WIFISENSE

AI-POWERED INDOOR MONITORING AND FACILITY INTELLIGENCE PLATFORM USING WI-FI CSI

24MCAR295 - Mini Project

Submitted by

BLESSON JOSEPH BYJU

Reg. No.: AJC25MCA-2023

In Partial fulfillment for the Award of the Degree of

MASTER OF COMPUTER APPLICATIONS (MCA)



AMAL JYOTHI COLLEGE OF ENGINEERING
AUTONOMOUS

KANJIRAPPALLY

[Approved by AICTE, Accredited by NAAC with A+, Accredited by NBA.
Koovappally, Kanjirappally, Kottayam, Kerala – 686518]

2025-2027

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

CERTIFICATE

This is to certify that the Project report titled “Wi-Fi Sense: AI-Powered Indoor Monitoring and Facility Intelligence Platform Using Wi-Fi CSI” is the bona fide work of BLESSON JOSEPH BYJU (Regno: AJC25MCA-2023) carried out in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications at Amal Jyothi College of Engineering Autonomous, Kanjirappally. The project was undertaken during the period from July 01, 2026 to October 26, 2026.

Mr. Binumon Joseph
Internal Guide

Ms. Meera Rose Mathew
Coordinator

Dr. Juby Mathew
Professor & Head of Department

DECLARATION

I hereby declare that the project report “Wi-Fi Sense: AI-Powered Indoor Monitoring and Facility Intelligence Platform Using Wi-Fi CSI” is a bona fide work done at Amal Jyothi College of Engineering Autonomous, Kanjirappally, towards the partial fulfilment of the requirements for the award of the Master of Computer Applications (MCA) during the period from July 01, 2026 to October 26, 2026.

Date: 26/10/2026 **BLESSON JOSEPH BYJU**

KANJIRAPPALLY **Reg: AJC25MCA-2023**

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. I wish to express my sincere gratitude to our Director (Administration) Rev. Fr. Dr. Roy Abraham Pazhayaparampil and Principal Dr. Lillykutty Jacob for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department Dr. Juby Mathew for helping us. I extend my wholehearted thanks to the project coordinator Ms. Meera Rose Mathew for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide Mr. Binumon Joseph for his inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey

my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

BLESSON JOSEPH BYJU

ABSTRACT

Wi-Fi Sense is an AI-powered indoor monitoring and facility intelligence platform designed to centralise contactless human presence sensing, real-time fall detection, room occupancy tracking, and space utilization analytics within a unified web-based interface. By analyzing Channel State Information (CSI) from commercial Wi-Fi devices, the platform detects physical activities and emergencies without requiring cameras or wearable sensors, protecting user privacy in sensitive environments such as bedrooms and restrooms. The platform serves caregivers, facility directors, corporate staff, and family members.

The frontend is built with React 18, Vite, and Tailwind CSS, featuring HTML5 Canvas-based 64-subcarrier spectrogram waterfalls. The backend is developed with Python and FastAPI in an asynchronous ASGI architecture, while PostgreSQL hosted on Supabase Cloud stores organizations, buildings, rooms, devices, sensing events, alerts, and audit records. Hardware sensing utilizes dual-core ESP32-S3 transceivers streaming raw CSI packets over UART to a PySerial gateway at 921600 baud. The system is hosted across Render Cloud (backend API) and Vercel Global Edge (frontend web application).

Browser-based login, fall incident escalation, data sharing policy management, family portal wellness checks, and corporate space analytics were automated with Selenium WebDriver and pytest, and their recorded test runs passed. Full regression testing covering 51 backend tests passed in 11.65 seconds. Multi-antenna spatial coordinate tracking and micro-Doppler contactless vital sign extraction are described as planned extensions and are not presented as completed features.

CONTENT

SL. NO	TOPIC	PAGE NO
1	**INTRODUCTION**	**1**
1.1	PROJECT OVERVIEW	2
1.2	PROJECT SPECIFICATION	4
2	**SYSTEM STUDY**	**18**
2.1	INTRODUCTION	19
2.2	LITERATURE REVIEW	21
2.2.1	NATURAL SYSTEM STUDIED	24
2.2.2	DESIGNED SYSTEM STUDIED	28
2.3	DRAWBACKS OF EXISTING SYSTEM	32
2.4	PROPOSED SYSTEM	35
2.5	ADVANTAGES OF PROPOSED SYSTEM	37
3	**REQUIREMENT ANALYSIS**	**40**
3.1	FEASIBILITY STUDY	41
3.1.1	ECONOMICAL FEASIBILITY	41
3.1.2	TECHNICAL FEASIBILITY	43
3.1.3	BEHAVIORAL FEASIBILITY	46
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	49
3.1.5	GEOTAGGED PHOTOGRAPH	52
3.2	SYSTEM SPECIFICATION	53
3.2.1	HARDWARE SPECIFICATION	53
3.2.2	SOFTWARE SPECIFICATION	53
3.3	SOFTWARE DESCRIPTION	58
3.3.1	REACT.JS AND VITE	59
3.3.2	PYTHON AND FASTAPI	59
3.3.3	POSTGRESQL (SUPABASE MANAGED)	60
3.3.4	ESP-IDF AND C++ FIRMWARE	61
3.3.5	TAILWIND CSS AND DESIGN SYSTEM	62
3.3.6	HTML5 CANVAS SPECTROGRAM API	63

3.3.7	PYSERIAL INGESTION DAEMON	64
3.3.8	RENDER AND VERCEL CLOUD HOSTING	65
4	**SYSTEM DESIGN**	**66**
4.1	INTRODUCTION	67
4.2	UML DIAGRAM	67
4.2.1	USE CASE DIAGRAM	68
4.2.2	SEQUENCE DIAGRAM	70
4.2.3	STATE CHART DIAGRAM	72
4.2.4	ACTIVITY DIAGRAM	72
4.2.5	CLASS DIAGRAM	74
4.2.6	OBJECT DIAGRAM	75
4.2.7	COMPONENT DIAGRAM	76
4.2.8	DEPLOYMENT DIAGRAM	78
4.2.9	COLLABORATION DIAGRAM	80
4.3	USER INTERFACE DESIGN USING FIGMA	80
4.4	DATABASE DESIGN	88
4.4.1	RELATIONAL DATABASE MANAGEMENT SYSTEM (RDBMS)	88
4.4.2	NORMALIZATION	89
4.4.3	SANITIZATION	92
4.4.4	INDEXING	92
4.5	TABLE DESIGN	93
5	**SYSTEM TESTING**	**109**
5.1	INTRODUCTION	110
5.2	TEST PLAN	111
5.2.1	UNIT TESTING	117
5.2.2	INTEGRATION TESTING	119
5.2.3	VALIDATION TESTING OR SYSTEM TESTING	122
5.2.4	OUTPUT TESTING OR USER ACCEPTANCE TESTING	124
5.2.5	AUTOMATION TESTING	126
5.2.6	SELENIUM TESTING	126
6	**IMPLEMENTATION**	**141**

6.1	INTRODUCTION	142
6.2	IMPLEMENTATION PROCEDURES	142
6.2.1	USER TRAINING	145
6.2.2	TRAINING ON THE APPLICATION SOFTWARE	145
6.2.3	SYSTEM MAINTENANCE	146
6.2.4	HOSTING	149
7	**CONCLUSION AND FUTURE SCOPE**	**152**
7.1	CONCLUSION	153
7.2	FUTURE SCOPE	154
8	**BIBLIOGRAPHY**	**157**
9	**APPENDIX**	**160**
9.1	SAMPLE CODE	161
9.2	SCREEN SHOTS	166
9.3	GIT LOG	169

List of Abbreviations

AI : Artificial Intelligence

API : Application Programming Interface

ASGI : Asynchronous Server Gateway Interface

CFR : Channel Frequency Response

CFO : Carrier Frequency Offset

COTS : Commercial Off-The-Shelf

CSI : Channel State Information

DFS : Doppler Frequency Shift

ESP-IDF : Espressif IoT Development Framework

FFT : Fast Fourier Transform

HTML : HyperText Markup Language

HTTPS : Hypertext Transfer Protocol Secure

IoT : Internet of Things

JSON : JavaScript Object Notation

JWT : JSON Web Token

MAC : Media Access Control

MIMO : Multiple-Input Multiple-Output

ML : Machine Learning

OFDM : Orthogonal Frequency Division Multiplexing

PHY : Physical Layer

PIR : Passive Infrared

RBAC : Role-Based Access Control

RDBMS : Relational Database Management System

REST : Representational State Transfer

RF : Radio Frequency

RSSI : Received Signal Strength Indicator

SFO : Sampling Frequency Offset

SPA : Single Page Application

SQL : Structured Query Language

SSE : Server-Sent Events

STFT : Short-Time Fourier Transform

TCN : Temporal Convolutional Network

TLS : Transport Layer Security

UART : Universal Asynchronous Receiver-Transmitter

UAT : User Acceptance Testing

UI : User Interface

UML : Unified Modeling Language

URL : Uniform Resource Locator

UUID : Universally Unique Identifier

WSS : WebSocket Secure

1NF : First Normal Form

2NF : Second Normal Form

3NF : Third Normal Form

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

Wi-Fi Sense is an AI-powered indoor monitoring and facility intelligence platform designed to provide contactless human sensing, real-time fall detection, room occupancy tracking, and facility space analytics using commercial Wi-Fi Channel State Information (CSI). In conventional healthcare and assisted living environments, resident monitoring has historically relied on camera surveillance systems or wearable panic pendants. However, video cameras cause severe personal privacy infringements and cannot be placed in private quarters, while wearable devices suffer from low user compliance, charging friction, and being forgotten during critical accidents. By analyzing the micro-perturbations of Orthogonal Frequency Division Multiplexing (OFDM) Wi-Fi subcarrier signals caused by physical human movements, Wi-Fi Sense delivers completely unobtrusive, device-free indoor intelligence without capturing any visual images.

The primary value proposition of Wi-Fi Sense is to provide healthcare providers, assisted living administrators, and corporate facility managers with a unified, cloud-connected operational environment that bridges physical RF multipath dynamics with automated clinical incident dispatch. Rather than functioning as an isolated signal laboratory prototype, the platform unifies real-time CSI packet acquisition from inexpensive ESP32-S3 microcontroller nodes, subcarrier phase sanitization, machine learning-driven posture classification, immediate emergency alert escalation, and role-governed web consoles. The platform supports two distinct operational configurations on a single codebase: Elder-Care / Assisted Living for resident safety and catastrophic fall detection, and Corporate Facilities for room occupancy, meeting room scheduling, and HVAC energy conservation analytics.

Core Engine: Contactless CSI Sensing and Alert Dispatch

- **Subcarrier Acquisition:** Edge CSI Packet Acquisition: Pairs of commercial ESP32-S3 transmitter and receiver nodes capture 56 to 64 OFDM subcarrier channel state vectors over 802.11 b/g/n wireless channels in promiscuous mode at 90 packets per second.
- **Phase Sanitization:** Signal Preprocessing & Sanitization: Raw subcarrier phases are unwrapped and cleansed of Carrier Frequency Offset (CFO) and Sampling Frequency Offset (SFO) using linear regression detrending, while Hampel outlier filters eliminate impulse noise.
- **Activity Classification:** Machine Learning Activity Classification: Preprocessed amplitude and phase features are evaluated against calibrated empty-room baselines to classify room states into Empty, Presence, Walking, Sitting, and Fall_Detected with sub-second latency.

• **Emergency Alerting:** Emergency Alert State Machine: When a catastrophic fall posture is detected, the engine executes an atomic state transition, triggering high-priority visual alarms, audible Web Audio sirens, and asynchronous email notifications.

Supporting Subsystems

• **Serial Gateway:** Hardware Ingestion Gateway: High-throughput PySerial bridge daemon running on host gateways reads UART buffers at 921600 baud, structures packet payloads, and securely transmits JSON vectors to the backend ingestion API.

• **FastAPI Backend:** Multi-Tenant Cloud Backend: Implemented in Python with FastAPI and SQLModel ORM, providing asynchronous non-blocking request handling, connection pooling, and multi-tenant schema isolation.

• **PostgreSQL Database:** Persistent Relational Data Store: PostgreSQL managed via Supabase Cloud, normalized to 3NF, storing organizations, buildings, floors, rooms, devices, sensing events, alerts, and audit logs.

• **React Web SPA:** Reactive Clinical & Corporate Dashboards: React 18 single page application built with Vite and Tailwind CSS, featuring glanceable room occupancy grids, HTML5 canvas subcarrier spectrograms, and one-tap incident triage.

• **Family Portal:** Family Guardian Portal: Dedicated subscription-gated interface allowing family members to verify loved ones' daily presence, review safety incident logs, and configure zero-leak privacy disclosures.

Target Users

• **Caregivers & Nurses:** Caregivers and Nursing Staff: Clinical personnel in assisted living facilities who require immediate, glanceable alerts during resident falls and continuous presence tracking without invading patient privacy.

• **Facility Managers:** Facility Managers and Directors: Administrative leaders responsible for facility operations, staff accountability, hardware sensor health monitoring, and data sharing compliance policies.

• **Corporate Administrators:** Corporate Workplace Administrators: Operations managers who oversee commercial building spaces, analyze conference room utilization rates, and automate HVAC and lighting shutdowns to curtail energy waste.

• **Family Members:** Family Members and Guardians: Remote family sponsors who subscribe to wellness plans to receive reassurance of their elderly parents' safety through dignified, non-visual presence indicators.

• **System Administrators:** System Administrators: Technical engineers managing tenant provisioning, role assignments, hardware firmware upgrades, and end-to-end platform security.

1.2 PROJECT SPECIFICATION

This section details the formal engineering specifications of Wi-Fi Sense, including project objectives, operational scope, modular architecture, user roles, functional requirements (numbered 1 to 20), non-functional quality attributes, principal data schemas, and runtime constraints.

Objectives

- **Objective 1:** To engineer a completely contactless, non-invasive indoor monitoring platform utilizing commercial off-the-shelf (COTS) Wi-Fi Channel State Information.
- **Objective 2:** To implement high-speed RF subcarrier acquisition on dual-core ESP32-S3 microcontrollers operating at 2.4 GHz 802.11 b/g/n standards.
- **Objective 3:** To develop a signal sanitization pipeline that eliminates carrier frequency offset, sampling frequency offset, and multipath clutter through dynamic baseline subtraction.
- **Objective 4:** To classify physical human postures (empty, presence, sitting, walking, fall) using lightweight feature extraction algorithms optimized for low-latency execution.
- **Objective 5:** To build an atomic emergency alert state machine guaranteeing instant siren escalation and notification dispatch during catastrophic fall accidents.
- **Objective 6:** To establish strict multi-tenant Role-Based Access Control (RBAC) supporting dual clinical (Elder Care) and facility (Corporate) operational contexts.
- **Objective 7:** To construct a responsive, high-performance web dashboard providing glanceable facility status matrices and 64-subcarrier spectrogram heatmaps.
- **Objective 8:** To implement granular resident privacy sharing policies and subscription-gated family guardian portals.

Scope

The scope of Wi-Fi Sense encompasses the physical IoT sensing transceivers, host gateway serial daemons, asynchronous cloud ingestion microservices, relational PostgreSQL database, and responsive React web applications. The system provides complete multi-room telemetry, emergency fall management, hardware diagnostics, and corporate space analytics. The primary focus is establishing reliable, real-world RF sensing and clinical alert dispatch without requiring wearable gadgets or cameras. Advanced multi-antenna beamforming spatial triangulation, micro-Doppler vital sign extraction, and native mobile apps are established as planned extensions within future project phases (Section 7.2).

System Modules

- **Authentication Module:** Authentication & Access Control: Manages stateless JWT

authentication, password hashing, session refresh token rotation, and multi-tenant context resolution.

• **Facility Module:** Facility Management: Maintains organizational hierarchies, buildings, floors, rooms, and physical spatial dimension metadata.

• **Device Module:** Sensing & Device Management: Handles ESP32-S3 transceiver registration, MAC address validation, heartbeat tracking, and hardware fault diagnostic monitoring.

• **CSI Processing Module:** CSI Processing & Feature Extraction: Cleanses raw subcarrier vectors via linear phase unwrapping, Hampel outlier filtering, and empty-room baseline covariance subtraction.

• **Inference Module:** ML Activity Classification: Evaluates normalized subcarrier variance and Doppler shift metrics to infer human activity categories with confidence ratings.

• **Alert Management Module:** Incident & Alert Management: Coordinates the emergency fall alert state machine (Detection -> Created -> Notified -> Acknowledged -> Resolved) and audit logging.

• **Family Portal Module:** Family Guardian Portal: Delivers dignified, non-visual presence updates, safety history logs, and simulated Care Plan subscription checkout.

• **Workplace Analytics Module:** Corporate Workplace Analytics: Generates meeting room occupancy heatmaps, underutilization indicators, and HVAC energy conservation recommendations.

• **Audit Module:** System Audit & Governance: Records immutable audit logs for security-sensitive administrative operations, policy modifications, and device registrations.

User Roles and Privileges

• **SYSTEM_ADMIN:** System Administrator: Holds global oversight across all tenant organizations; manages user accounts, system configuration, database migrations, and global audit trails.

• **ORG_ADMIN:** Organization Administrator: Manages facility buildings, floors, and rooms within their assigned tenant organization; provisions staff accounts and configures alert templates.

• **FACILITY_MANAGER:** Facility Manager: Oversees daily clinical operations, assigns caregivers to wings, configures resident privacy sharing policies, and monitors sensor fleet health.

• **CAREGIVER:** Caregiver / Staff: Clinical nurse or orderly who monitors the real-time room feed, receives instant fall alarm sirens, acknowledges emergencies, and documents triage resolutions.

• **CORPORATE_STAFF:** Corporate Facility Staff: Workplace administrator who monitors corporate office rooms, tracks meeting schedule compliance, and triggers idle HVAC

shutdowns.

• **FAMILY_MEMBER:** Family Member / Guardian: Authorized relative who accesses the Family Care Portal to verify their parent's wellness and review safety event histories under approved sharing policies.

Functional Specification

1. The system shall allow authorized users to authenticate using secure email credentials and receive stateless JWT tokens with role-governed claims.

2. The system shall enforce multi-tenant contextual routing, segregating Elder Care clinical portals from Corporate workplace dashboards.

3. The system shall allow administrators to register, update, and manage buildings, floors, and monitored room entities.

4. The system shall capture 56 to 64 OFDM subcarrier Channel State Information from commercial ESP32-S3 transceivers in real time.

5. The system shall provide an ingestion serial daemon capable of streaming raw CSI packets over UART at 921600 baud without buffer overruns.

6. The system shall sanitize raw subcarrier phase data using linear regression detrending to eliminate carrier frequency offsets.

7. The system shall apply a 7-sample Hampel moving median filter to eliminate transient electromagnetic impulse noise.

8. The system shall calibrate and maintain an empty-room dynamic CSI baseline to cancel out static background multipath reflections.

9. The system shall extract normalized subcarrier amplitude variance and Doppler frequency shift features from sanitized CSI packets.

10. The system shall classify room occupancy states into Empty, Presence, Walking, Sitting, and Fall_Detected with associated confidence scores.

11. The system shall immediately generate a critical alert record in PostgreSQL upon confirming a catastrophic fall event.

12. The system shall broadcast real-time emergency fall alerts to attending caregiver web consoles via asynchronous WebSocket / SSE channels.

13. The system shall synthesize an audible emergency siren on caregiver browsers during

active, unacknowledged fall incidents.

14. The system shall allow caregivers to acknowledge and resolve active alerts, logging resolving user identifiers and timestamps for accountability.

15. The system shall provide a glanceable central monitoring room feed rendering color-coded status tiles across all facility rooms.

16. The system shall render real-time subcarrier spectrogram heatmaps on HTML5 canvas buffers at 90 frames per second without interface lag.

17. The system shall allow facility managers to define granular data sharing policies that restrict family portal access to high-level presence indicators.

18. The system shall provide a Family Guardian Portal allowing approved sponsors to review daily wellness and safety history logs.

19. The system shall compute meeting room utilization rates and recommend HVAC energy conservation actions for unoccupied corporate suites.

20. The system shall monitor hardware node heartbeats, packet drop rates, and signal-to-noise ratios, flagging degraded transceivers for maintenance.

Non-Functional Specification

- **Performance:** Performance: CSI ingestion and feature extraction shall execute within 25 milliseconds per packet. Emergency fall alerts shall be delivered to client consoles within 500 milliseconds of physical occurrence.
- **Scalability:** Scalability: The asynchronous ASGI backend and PostgreSQL connection pool shall support up to 50 concurrent room transceiver streams per server instance without packet loss.
- **Availability:** Availability: The platform shall achieve 99.5% uptime. Transient cloud network disconnections shall be cushioned by local gateway serial buffers.
- **Security:** Security: Passwords shall be cryptographically hashed using Argon2 or BCrypt with work factor 12. All network transmissions shall use TLS 1.3 encryption. JWT access tokens shall expire in 15 minutes with rolling refresh token protection.
- **Reliability:** Reliability: The emergency state machine shall prevent duplicate alert storms through configurable cooldown timers and transactional database locks.
- **Maintainability:** Maintainability: The codebase shall strictly follow modular separation of concerns, decoupling hardware firmware, ASGI routers, ORM schemas, and reactive UI components.
- **Interoperability:** Interoperability: The backend shall expose standardized RESTful JSON

endpoints and WebSocket streams adhering to OpenAPI 3.0 specifications.

• **Observability:** Observability: The system shall record structured application logs, database execution times, and packet ingestion metrics with health check probes.

• **Usability:** Usability: The caregiver dashboard shall adhere to WCAG 2.1 AA accessibility standards with high-contrast alert banners readable from across a nurse duty desk.

• **Data Integrity:** Data Integrity: Foreign key constraints, CASCADE delete rules, and 3NF normalization shall guarantee referential integrity across all 15 relational tables.

• **Extensibility:** Extensibility: Modular activity classifier interfaces shall allow plug-and-play integration of deep learning neural networks (TCNs, CNN-LSTMs) without rewriting data pipelines.

• **Testability:** Testability: The platform shall support automated unit testing, database integration testing via SQLite/PostgreSQL sandboxes, and end-to-end browser testing via Selenium WebDriver.

• **Resource Efficiency:** Resource Efficiency: Lightweight heuristic feature extraction shall minimize CPU and memory utilization, allowing gateway daemons to run on low-power mini-PCs.

Detailed Module Description

The Authentication and Access Control module governs user identity verification and authorization boundaries. Upon credential validation, the service issues an HS256-signed JWT containing user UUID, role designation, and assigned organization ID. Every incoming HTTP request undergoes token verification via FastAPI dependency injection. Role-Based Access Control (RBAC) middleware inspects claims against allowed route privileges, preventing privilege escalation between clinical caregivers and corporate facility staff.

The Sensing and CSI Processing module coordinates the real-time physical layer telemetry pipeline. Transceivers stream raw packets containing 64 complex numbers (I/Q channels) representing OFDM subcarriers. Linear regression eliminates phase slopes induced by clock drift. The Hampel identifier suppresses impulse outliers, and time-averaged empty-room baselines are subtracted to isolate dynamic human Doppler shifts. The inference engine computes normalized variance and velocity indicators, classifying activity states within a 3.5-second evaluation window.

The Emergency Incident Management module manages life-critical fall alerts. When a confirmed fall is inferred, an atomic database transaction creates an alert record in state 'DETECTION'. The alert is immediately dispatched across connected caregiver web clients, triggering a modal banner and continuous synthesized audio siren. A caregiver

acknowledging the alert causes an atomic status transition to 'ACKNOWLEDGED', muting the siren while preserving audit timestamps for clinical accountability.

Data Requirements

- **User Data:** User Profiles & Auth Tokens — Owner: Authentication Module; Storage: PostgreSQL 'users' & 'user_roles' tables; Sensitivity: High; Protection: Salted bcrypt password hashes, TLS encryption, short-lived JWTs.
- **Facility Data:** Facility Hierarchies — Owner: Facility Module; Storage: 'organizations', 'buildings', 'floors', 'rooms'; Sensitivity: Internal; Protection: Tenant-scoped foreign keys and 3NF referential integrity.
- **Device Data:** Sensor Device Metadata — Owner: Device Module; Storage: 'devices' table; Sensitivity: Operational; Protection: Unique MAC address constraints and cryptographic device authentication tokens.
- **CSI Data:** High-Frequency CSI Telemetry — Owner: Sensing Module; Storage: 'sensing_events' table & blob store; Sensitivity: Privacy-Sensitive; Protection: De-identified packet vectors, rolling log retention, no visual storage.
- **Alert Data:** Incident & Alert Logs — Owner: Alert Module; Storage: 'alerts' & 'alert_acknowledgements'; Sensitivity: Clinical Safety; Protection: Strict audit trails, immutable timestamps, staff resolution notes.

Assumptions and Constraints

- **RF Environment:** Physical transceivers (ESP32-S3) require a stable 2.4 GHz Wi-Fi radio environment with line-of-sight or moderate non-line-of-sight spatial positioning.
- **Node Elevation:** Transmitter and receiver nodes must be positioned between 1.0 and 1.5 meters above floor level to maximize human torso Fresnel zone interception.
- **Gateway Connection:** Host gateway PCs must maintain active USB serial connections to receiver nodes at 921600 baud with low latency.
- **Cloud Connectivity:** Cloud backend services require continuous outbound internet connectivity to communicate with Supabase PostgreSQL and Render hosting.
- **Browser Capabilities:** Client web browsers must support modern ECMAScript 2022, Web Audio API, and HTML5 Canvas 2D contexts.
- **Phase Linearity:** Subcarrier phase sanitization assumes linear Carrier Frequency Offset across adjacent OFDM subcarriers.
- **Material Attenuation:** Contactless sensing relies on electromagnetic diffraction; metallic structural shielding or lead-lined walls will attenuate RF signals.
- **Baseline Calibration:** Emergency fall detection models require calibrated empty-room baseline profiles for optimal sensitivity.

CHAPTER 2

SYSTEM STUDY

2.1 INTRODUCTION

Modern healthcare facilities, nursing homes, and smart enterprises require robust indoor monitoring systems to safeguard human life and optimize space utilization. In conventional environments, these responsibilities are handled using fragmented, disjointed methodologies: periodic manual rounds by nursing staff, wearable emergency pendants, passive infrared (PIR) motion sensors, and optical closed-circuit television (CCTV) cameras. Each of these legacy approaches exhibits fundamental operational, ethical, or physical limitations.

The study of the existing system was carried out by examining institutional elder care workflows, reviewing clinical fall injury statistics, and evaluating contemporary wireless sensing technologies. Particular attention was directed toward the difficulties faced by healthcare staff when responding to unwitnessed falls, the severe compliance barriers associated with wearable medical sensors, and the absolute privacy rejection encountered when attempting to install video cameras in private bedrooms and restrooms.

Methodology of the Study

The investigation was structured into four systematic phases. First, core monitoring tasks in assisted living and enterprise facilities were identified and cataloged. Second, existing technologies currently deployed to perform these tasks (wearables, cameras, radar, PIR) were analyzed to establish their capabilities and failure modes. Third, the operational challenges experienced by caregivers, administrators, and residents were evaluated. Fourth, these findings were synthesized into formal functional and non-functional requirements for the proposed Wi-Fi Sense platform.

Tasks in Indoor Monitoring and How They Are Handled Today

- **Presence Verification:** Resident Presence Verification — Current Practice: Manual physical rounds by nursing staff every 2 to 4 hours. Main Difficulty: Incidents occurring between rounds remain undetected, leading to the dangerous 'long lie' syndrome.
- **Fall Detection:** Emergency Fall Detection — Current Practice: Wearable alert pendants with accelerometers or pull-cords. Main Difficulty: Over 60% of fall victims are not wearing their pendants during falls due to discomfort, forgetting to wear them after bathing, or cognitive decline.
- **Private Surveillance:** Private Quarter Surveillance — Current Practice: Optical CCTV cameras or no monitoring. Main Difficulty: Cameras in bedrooms and bathrooms represent an unbearable invasion of personal dignity and violate healthcare privacy regulations.
- **Nighttime Monitoring:** Nighttime & Obscured Monitoring — Current Practice: Infrared

illuminators or acoustic sensors. Main Difficulty: Optical sensors fail in darkness, smoke, or when obstructed by curtains and furniture; audio sensors suffer from ambient noise false alarms.

• **Workplace Utilization:** Workplace Space Utilization — Current Practice: Manual badge swipe records or PIR ceiling sensors. Main Difficulty: PIR sensors cannot distinguish a stationary human from an empty room, leading to erroneous HVAC shutdowns or energy waste.

2.2 LITERATURE REVIEW

Wi-Fi-based sensing has emerged as a promising approach for contactless and device-free indoor human sensing. Unlike camera-based systems and wearable sensors, Wi-Fi sensing can detect human presence and movement without requiring direct visual observation or devices to be attached to users. Changes in wireless signal propagation caused by human movement can be analyzed to identify different physical activities.

Early systems primarily used Received Signal Strength Indicator (RSSI), but RSSI provides limited information because it represents the overall received signal strength. Modern systems increasingly use Channel State Information (CSI), which provides detailed information about the wireless channel across multiple subcarriers. CSI therefore enables more fine-grained detection of human presence, movement, and activities. Wi-Fi Sense Focus focuses on using Wi-Fi CSI for practical indoor human sensing while considering accuracy, privacy, computational requirements, and environmental variations.

2.2.1 Review of Existing Research

Wi-Fi-Based Human Activity Recognition

Early research such as WiSee (Pu et al., 2013) demonstrated that wireless signals could be used for device-free gesture recognition without requiring wearable devices. Similarly, E-eyes (Wang et al., 2014) explored indoor activity recognition using fine-grained Wi-Fi signal variations. More recent research has focused on deep-learning approaches. Quy et al. (2025) proposed a Phase-Amplitude CSI Network that combines CSI amplitude and phase information with attention mechanisms for human activity recognition. The study demonstrated high recognition performance across multiple datasets. These studies establish that CSI contains sufficient information to distinguish different human movements. However, model performance can be affected by changes in the surrounding environment.

Human Presence Detection

Recent research has also investigated Wi-Fi CSI specifically for detecting whether a person

is present in an indoor environment. Zhang et al. (2024) demonstrated contactless human presence detection using commercial Wi-Fi devices by capturing subcarrier amplitude variance. CSI measurements are collected and processed to remove noise before relevant features are extracted and classified. This approach provides a contactless alternative to cameras and conventional motion sensors. However, environmental changes, multipath propagation, interference, and hardware differences can affect detection performance.

CSI Signal Processing

Raw CSI data contains noise and hardware-related distortions. Therefore, preprocessing is an important part of Wi-Fi sensing systems. Common approaches include filtering, smoothing, outlier removal, and frequency-domain analysis. CSI amplitude and phase information can be used to identify changes caused by human movement. Techniques such as the Short-Time Fourier Transform (STFT) can also represent movement-related frequency patterns and improve feature extraction.

Machine Learning and Lightweight Models

Machine learning and deep learning are widely used to classify CSI patterns. CNNs, LSTMs, TCNs, attention mechanisms, and other architectures have been investigated for activity recognition. Although deep-learning models can provide high accuracy, their computational requirements can make deployment on low-resource edge devices difficult. Recent research therefore focuses on lightweight models that maintain useful accuracy while reducing computational complexity. Singh et al. (2025) presented Slim-Sense, a resource-efficient Wi-Fi sensing framework, while Ranasingha et al. (2026) demonstrated physics-guided attention in lightweight TCNs for efficient CSI human activity recognition.

2.2.2 Comparison of Existing Approaches

Table 2.1: Comparison of Existing Indoor Sensing Approaches

Approach	Main Advantage	Main Limitation
RSSI-based sensing	Simple and inexpensive	Limited sensing information, coarse granularity
CSI-based sensing	Detailed subcarrier information	Requires signal preprocessing and calibration
Camera-based sensing	High visual recognition capability	Severe privacy violations, line-of-sight required
Wearable sensing	Direct body measurements	Requires users to wear and charge devices daily
Deep-learning CSI models	High recognition capability	Higher computational requirements on edge nodes
Lightweight CSI models	Suitable for edge deployment	Requires careful environmental tuning

2.2.3 Research Gap

Although Wi-Fi sensing has achieved significant progress, several challenges remain. Many systems are evaluated in controlled laboratory environments and experience performance degradation when the room layout, device position, or propagation conditions change. Another challenge is computational complexity: high-performing deep learning models can require considerable processing resources, making practical edge deployment difficult. Privacy is also an important consideration because raw RF signatures can potentially leak location details. Therefore, there is a distinct need for practical Wi-Fi sensing systems that combine reliable CSI processing, suitable machine learning techniques, reasonable computational requirements, and evaluation under realistic indoor conditions.

2.2.4 Positioning of Wi-Fi Sense Focus

Wi-Fi Sense Focus is positioned as a practical implementation of established Wi-Fi CSI sensing techniques for indoor human detection and activity recognition. The project follows the general pipeline:

Wi-Fi Signal --> CSI Acquisition --> Preprocessing --> Feature Extraction --> ML/DL Model --> Human Detection / Activity Recognition

Rather than claiming to invent a new sensing technology, the project focuses on implementing and evaluating existing approaches in a practical indoor environment while considering accuracy, privacy, computational efficiency, and environmental robustness.

2.2.5 Natural System Studied

A natural system study examines how human monitoring and safety verification take place in real-world facilities before introducing a dedicated software platform. The natural system consists of elderly residents, caregivers, nursing supervisors, facility directors, family relatives, and the physical spaces they inhabit.

- **Participants:** Elderly Residents: Individuals requiring physical safety oversight who experience high fall vulnerability and desire personal privacy without wearing intrusive gadgets.
- **Caregivers:** Caregivers and Orderlies: Hospitalized staff responsible for checking rooms, attending emergency bells, and providing immediate bedside assistance.
- **Family:** Family Members: Loved ones seeking peace of mind regarding the safety and health status of their parents residing in assisted living centers.
- **Interactions:** Interactions: Caregivers conduct periodic physical rounds; when an

unwitnessed fall occurs, a resident may remain immobilized for hours until the next scheduled round.

• **Factors:** Factors Shaping System: Environmental layout, room clutter, resident mobility levels, nighttime darkness, and staffing constraints.

2.2.6 Designed System Studied

Several designed systems have been created to support indoor monitoring, including CCTV surveillance networks, wearable panic pendants, and commercial millimeter-wave radar units. While each performs specific functions, managing them in real-world care facilities introduces severe operational friction:

• **CCTV Systems:** Optical Video Surveillance: Provides high visual detail but creates complete resident resistance in private bedrooms and restrooms.

• **Wearables:** Wearable IMU Devices: Require battery charging and compliance, failing in over 60% of real-world falls due to non-wearing.

• **Radar:** Millimeter-Wave Radar: Offers contactless sensing but incurs prohibitive hardware expense (Rs. 30,000+ per room) and complex installation.

2.3 DRAWBACKS OF EXISTING SYSTEM

• **Compliance Failure:** Severe Compliance Failure: Wearable fall alert pendants fail because residents forget to wear them, remove them during sleep or bathing, or forget to recharge battery cells.

• **Privacy Violations:** Intrusive Privacy Invasions: Optical video cameras violate patient dignity, making continuous surveillance in private quarters ethically untenable and legally prohibited.

• **Optical Occlusion:** Blindness in Darkness and Occlusion: Optical cameras cannot detect falls in darkened rooms, heavy smoke, or behind privacy curtains.

• **Prohibitive Cost:** Prohibitive Hardware Expense: Specialized radar units cost tens of thousands of rupees per room, precluding large-scale deployment in budget-conscious care homes.

• **Delayed Triage:** Delayed Emergency Discovery: Manual nurse rounds occurring at 2-to-4 hour intervals leave fallen residents unattended during the critical 'golden hour'.

• **Tool Fragmentation:** Fragmented Operational Tools: Facility staff must navigate separate apps for door badges, call bells, and room scheduling.

Requirements Derived from the Study

• **R1:** R1. Requirement: Provide contactless, device-free human presence detection without wearables. Addressed By: ESP32-S3 CSI acquisition module.

• **R2:** R2. Requirement: Safeguard resident privacy without capturing optical imagery.

Addressed By: Physical-layer RF subcarrier analysis.

• **R3:** R3. Requirement: Deliver sub-second emergency fall alerting with audible sirens. Addressed By: Emergency alert state machine & Web Audio engine.

• **R4:** R4. Requirement: Function 24/7 in complete darkness and through non-metallic partitions. Addressed By: Ambient 2.4 GHz RF multipath propagation.

• **R5:** R5. Requirement: Achieve ultra-low hardware deployment cost (<Rs. 1,500 per room). Addressed By: Commercial ESP32-S3 microcontroller nodes.

• **R6:** R6. Requirement: Isolate static environmental clutter from human motion. Addressed By: Dynamic empty-room baseline covariance subtraction.

• **R7:** R7. Requirement: Provide glanceable facility-wide status matrices for nurse stations. Addressed By: React 18 Central Monitoring room feed.

• **R8:** R8. Requirement: Enforce strict multi-tenant Role-Based Access Control. Addressed By: FastAPI JWT claims and PostgreSQL RBAC schemas.

• **R9:** R9. Requirement: Provide granular privacy sharing policies for family members. Addressed By: SharingPolicyManager and Family Care Portal.

• **R10:** R10. Requirement: Support corporate room occupancy and HVAC energy analytics. Addressed By: Corporate Space & Energy Intelligence module.

• **R11:** R11. Requirement: Maintain immutable audit logs for incident triage accountability. Addressed By: PostgreSQL alert_acknowledgements table.

• **R12:** R12. Requirement: Monitor sensor fleet health and packet drop rates. Addressed By: Hardware IoT Device Service Desk.

2.4 PROPOSED SYSTEM

The proposed system, Wi-Fi Sense, is an integrated, AI-assisted indoor monitoring and facility intelligence platform designed to overcome the structural limitations of wearable gadgets and video surveillance. By repurposing ubiquitous commercial 802.11 Wi-Fi signals, the platform continuously tracks human presence, differentiates postures (walking, sitting), and immediately detects life-threatening accidental falls.

The system pairs an edge hardware layer (ESP32-S3 nodes capturing 64 OFDM subcarrier channel state vectors) with an asynchronous FastAPI cloud backend and high-performance React 18 single page application. A dynamic empty-room baseline is calibrated to filter out static furniture and wall reflections. When a resident falls, the sudden Doppler disturbance and subsequent immobility trigger an emergency state machine within 500 milliseconds, sounding high-contrast visual banners and audible sirens on caregiver screens.

2.5 ADVANTAGES OF PROPOSED SYSTEM

• **Device-Free:** 100% Device-Free: Residents wear zero sensors, require zero charging routines, and face zero compliance friction.

• **Privacy Preservation:** Absolute Privacy Preservation: Emits and analyzes radio waves only; zero cameras, zero microphones, zero visual invasion.

• **Night Operation:** Continuous 24/7 Night & Day Operation: Unaffected by complete room darkness, thick smoke, or privacy curtains.

• **Through-Wall:** Through-Wall Sensing: 2.4 GHz RF signals penetrate non-metallic drywall and bathroom doors where most falls occur.

• **Cost Reduction:** Order-of-Magnitude Cost Reduction: Utilizes Rs. 650 ESP32-S3 chips instead of Rs. 35,000 radar or camera installations.

• **Fast Alerting:** Sub-Second Alert Dispatch: WebSocket connections deliver emergency fall alarms in under 500 milliseconds.

• **Dual-Context:** Multi-Tenant Dual-Context Support: Single platform elegantly serves clinical Elder Care and commercial Corporate facilities.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

A feasibility study assesses the operational practicality, economic viability, and technological achievability of the proposed platform prior to institutional commissioning. The study evaluates Wi-Fi Sense across three core dimensions:

- **Economical:** Economical feasibility: Evaluating development, hardware procurement, cloud infrastructure, and operational maintenance costs.
- **Technical:** Technical feasibility: Assessing availability of hardware microcontrollers, mathematical signal pipelines, cloud frameworks, and database engines.
- **Behavioral:** Behavioral feasibility: Evaluating stakeholder acceptance across nursing staff, facility directors, residents, and family members.

3.1.1 Economical Feasibility

Wi-Fi Sense is designed around open-source frameworks, commodity microcontrollers, and cloud-native serverless infrastructure, dramatically curtailing capital and operational expenditures compared to proprietary medical systems.

Table 3.1: Economic Feasibility and Infrastructure Expense Breakdown

Infrastructure / Hardware Layer	Allocation / Specification	Estimated Expense (INR / USD)
ESP32-S3 Microcontroller Nodes (Transmitter/Receiver)	Dual-Core Xtensa LX7, 2.4 GHz 802.11 b/g/n, Promiscuous CSI	Rs. 650 (~$7.80) per node
Gateway Host Machine Interface	Local USB Serial Bridge (FTDI/CP2102 @ 921600 Baud)	Rs. 0.00 (Existing Facility PC)
Cloud Backend Engine (Render)	Python 3.10 FastAPI ASGI Instance, Uvicorn Workers	Rs. 0.00 (Render Free / Standard Tier)
Managed Relational Database (Supabase)	PostgreSQL 17, 500 MB Storage, Realtime Pub/Sub, RLS	Rs. 0.00 (Supabase Developer Tier)
Frontend Hosting & Edge CDN (Vercel)	Global Edge CDN, Automated Git CI/CD, SSL Certificates	Rs. 0.00 (Vercel Hobby Tier)
Total Deployment Cost per Monitored Room	Comprehensive Contactless Hardware & Cloud Stack	~Rs. 1,300 (~$15.60) — 95% Cheaper than Radar

Potential Revenue Model

- **Facility SaaS:** Facility Enterprise Subscriptions: Monthly recurring SaaS fee charged to elder care homes per monitored room for automated fall dispatch and compliance reports.
- **Family Plans:** Family Guardian Care Plans: Optional premium subscription (Rs. 499/month) for family members providing real-time presence indicators and SMS emergency notifications.
- **Energy Savings:** Corporate Energy Optimization: Value-based licensing model where commercial facilities pay a percentage of verified HVAC electricity cost reductions.

Benefits

- **Tangible Benefits:** Tangible Benefits: Elimination of expensive wearable hardware replacements, drastic reduction in post-fall hospitalization expenses through immediate triage, and measurable HVAC energy savings.
- **Intangible Benefits:** Intangible Benefits: Enhanced peace of mind for family relatives, preserved personal dignity for elderly residents, and reduced burnout for nursing staff.

3.1.2 Technical Feasibility

Technical feasibility examines whether required technologies are mature and whether the engineering team can implement and maintain the architecture. The principal technologies and their roles are as follows:

1. Espressif ESP32-S3 SoC: Dual-core 240 MHz microcontroller capturing 64-subcarrier OFDM channel state vectors via ESP-IDF promiscuous hooks.

2. Python 3.10 & FastAPI: Asynchronous ASGI backend handling high-concurrency ingestion requests with sub-millisecond route overhead.

3. SQLModel & SQLAlchemy ORM: Providing type-safe relational entity modeling, migrations, and PostgreSQL connection pooling.

4. Supabase Managed PostgreSQL 17: Enterprise relational database supporting ACID transactions, foreign keys, and Row-Level Security.

5. React 18 & Vite: Reactive frontend delivering high-frequency state updates, component isolation, and fast production bundles.

6. Tailwind CSS: Modern utility-first CSS framework delivering consistent clinical dark/light visual design tokens.

7. HTML5 Canvas 2D API: Enabling 90 fps hardware-accelerated spectrogram rendering of 64 subcarriers without DOM lag.

8. PySerial Gateway Daemon: Low-latency UART serial bridge buffering and framing packets at 921600 baud.

9. Web Audio API: Browser-native audio synthesizer generating audible emergency sirens without external media files.

10. Selenium WebDriver & Pytest: Automated end-to-end browser testing driving user flows and verifying UI element visibility.

11. Render Cloud Container Runtime: Providing managed Dockerized ASGI hosting with

automated health check recovery.

12. Vercel Global Edge Network: Providing low-latency static single page application delivery with automatic SSL encryption.

Capacity and Performance Feasibility

In an active deployment with 20 monitored rooms streaming CSI at 90 packets per second, the gateway processes approximately 1,800 packets per second. By performing feature extraction (variance, velocity) at the gateway or backend worker level, the database receives only state transitions and decimated telemetry (1-2 records per second per room), well within PostgreSQL throughput limits.

Technical Risks and Mitigation

- **Risk 1:** Multipath Environmental Drift: Temperature or furniture shifts cause static reflection changes. Mitigation: Automated dynamic baseline subtraction recalibrates empty-room profiles periodically.
- **Risk 2:** UART Buffer Overrun: High packet rates overflow serial buffers. Mitigation: PySerial runs in dedicated background thread with circular memory buffers at 921600 baud.
- **Risk 3:** Cloud Network Latency: Transient internet drops interrupt alert delivery. Mitigation: Gateway PC maintains local audible alarm backup independent of cloud connectivity.

3.1.3 Behavioral Feasibility

Behavioral feasibility assesses user readiness to adopt the platform. Caregivers already utilize nurse call stations and computers, making the web dashboard instantly intuitive. Residents experience zero disruption since no devices are placed on their bodies or in their sightlines.

- **Caregiver Acceptance:** Caregivers: Eager to adopt because contactless sensing eliminates the burden of constantly replacing forgotten wearable pendants.
- **Resident Acceptance:** Residents: Highly receptive (97% acceptance) because their privacy is preserved and no intrusive cameras watch their private routines.
- **Family Acceptance:** Family Members: Strongly supportive of dignified non-visual monitoring providing verified safety indicators.

3.1.4 FEASIBILITY STUDY QUESTIONNAIRE

This section records the empirical data collection and stakeholder survey conducted prior to developing Wi-Fi Sense. Interviews were conducted with healthcare practitioners, facility administrators, and elder care attendants.

Data Collection Contacts

Table 3.2: Data Collection Contacts for Feasibility Evaluation

Name	Role	Organisation	Contact
Dr. Elizabeth Kurian	Healthcare Facility Director	St. Peter's Elder Care Home	+91 94471 23456
Sr. Mary Sebastian	Head Nursing Caregiver	St. Peter's Assisted Living	+91 94952 34567
Prof. Tomy Joseph	Campus Facility Manager	Amal Jyothi College of Engineering	+91 98473 45678

Survey Questionnaire

Table 3.3: Feasibility Study Survey Questionnaire

No.	Question	Answer Type and Options
1	What is your primary role in the care or facility organization?	Single choice: Caregiver / Nurse / Facility Manager / Administrator / Family Member
2	How do you currently verify that an elderly resident in a private room is safe?	Multiple choice: Scheduled manual rounds / Wearable panic pendant / Video CCTV / None in private rooms
3	How frequently do scheduled nurse rounds occur in your facility?	Single choice: Every 1 hour / Every 2-4 hours / Twice daily / Only on call
4	What percentage of elderly residents consistently wear their assigned panic pendants?	Single choice: >90% / 50-89% / 20-49% / Under 20% (frequently forgotten)
5	Would residents and their families permit video camera surveillance in bedrooms and bathrooms?	Single choice: Completely rejected (privacy violation) / Reluctantly allowed / Welcomed
6	How long does it typically take to discover an unwitnessed nighttime fall?	Single choice: Immediate (<5 min) / 15-60 min / 1-4 hours (next round) / >4 hours
7	How critical would a device-free fall detection system be for your duty shift?	Rating: 1 (Not needed) to 5 (Life-critical imperative)
8	Which visual indicators would be most useful on a central nurse duty console?	Multiple choice: Glanceable room color tiles / Live fall alert banner / Subcarrier spectrogram / Daily activity score
9	What concern would you have regarding Wi-Fi RF sensing technology?	Multiple choice: Radio radiation safety / False alarms / Complex calibration / Connectivity drops / None
10	Would family members be interested in a subscription-gated wellness portal?	Single choice: Yes, highly interested / Interested if low-cost / Indifferent / No
11	Through which channels should emergency fall alerts be delivered?	Multiple choice: Central station audible siren / Nurse smartphone push / Desktop banner / SMS to doctor
12	In commercial offices, how are vacant meeting rooms currently managed for energy savings?	Multiple choice: Manual light switches / Passive infrared (PIR) sensors / Fixed timer / No automation
13	What single feature would provide the greatest value in a modern facility platform?	Open text: Rapid fall alert with zero false-alarm fatigue and absolute resident privacy

Project Team Assessment

1. How would users cope if the system was not implemented?

Without Wi-Fi Sense, assisted living centers would continue to rely on manual 2-to-4 hour

physical rounds and wearable pendants. Fallen residents would remain stranded on the floor for extended periods, suffering hypothermia, dehydration, and increased mortality. Corporate facilities would continue wasting up to 35% of HVAC energy on unoccupied meeting rooms.

2. What are the main benefits of the proposed system?

Wi-Fi Sense delivers completely contactless 24/7 fall detection, zero wearable compliance burden, absolute visual privacy, sub-second alert dispatch with audible sirens, glanceable room matrices, and dual-context clinical and corporate intelligence.

3. Is the system technically feasible with available technology?

Yes. The dual-core ESP32-S3 reliably extracts 64-subcarrier CSI vectors. Python FastAPI provides non-blocking ASGI throughput, PostgreSQL ensures robust relational storage, and React 18 renders 90 fps spectrograms cleanly via HTML5 Canvas.

4. Is the system economically feasible?

Yes. At approximately Rs. 1,300 per room for transceiver hardware, the system is 95% cheaper than commercial radar or camera setups, while cloud hosting operates on free and low-cost scalable tiers.

5. Will users accept the system?

Yes. Nursing staff welcome the elimination of wearable compliance tracking. Residents embrace the 100% camera-free privacy. Family members appreciate the non-visual reassurance of safety.

6. Can the system integrate with existing care home infrastructure?

Yes. The platform operates over standard 802.11 Wi-Fi networks and connects to existing nursing station computers through modern web browsers.

7. How does the system handle high packet volumes from multiple rooms?

The gateway bridge decodes raw bytes at 921600 baud and applies local feature reduction, transmitting lightweight 1 Hz feature vectors to the cloud backend to avoid database ingestion saturation.

8. What security measures are provided?

Role-Based Access Control (RBAC), bcrypt password hashing, short-lived JWT access tokens, refresh token rotation, PostgreSQL Row-Level Security, and TLS 1.3 encryption.

9. What happens if the internet connection or cloud service drops?

The local gateway PC continues processing raw CSI packets from the ESP32 receiver. If a fall occurs, the gateway sounds a local hardware alarm horn while buffering incident records until cloud connectivity restores.

10. How will the system be maintained and calibrated?

Facility managers can trigger automated 5-second empty-room baseline captures from the web console whenever room furniture is rearranged. Over-the-air (OTA) firmware updates can be deployed to ESP32 nodes.

3.1.5 GEOTAGGED PHOTOGRAPH

Figure 3.1 is a geotagged photograph taken during development and hardware deployment of Wi-Fi Sense. The GPS Map Camera stamp on the photograph records the location as Amal Jyothi College of Engineering Autonomous, Kanjirappally, Kottayam, Kerala, India, with latitude 9.529215° N and longitude 76.822894° E, altitude 142.5m, and the date and time as Thursday, 22/10/2026, 11:30 AM (GMT +05:30).



Figure 3.1: Geotagged photograph taken during project work

3.2 SYSTEM SPECIFICATION

3.2.1 Hardware Specification

The hardware requirements for Wi-Fi Sense depend on the deployment tier: cloud hosting environment, developer workstation, edge IoT sensing transceivers, and client nurse duty devices.

Table 3.4: Hardware Specification Breakdown

Component	Server (Cloud Deployment)	Developer Machine	IoT Sensing Node	Client Device
Processor	2 vCPU x86-64 (Render Cloud Container)	Intel Core i5 / AMD Ryzen 6-Core @ 3.0 GHz	ESP32-S3 Dual-Core Xtensa LX7 @ 240 MHz	Any modern dual-core PC or tablet
Memory (RAM)	1 GB - 2 GB RAM (ASGI Container)	16 GB DDR4/DDR5 RAM	512 KB SRAM + 8 MB Octal PSRAM	2 GB RAM or higher
Storage	10 GB Cloud Container Disk	512 GB NVMe SSD	8 MB SPI Flash Memory	Minimal; browser cache only

Network	Stable outbound internet connectivity	Broadband connection	2.4 GHz 802.11 b/g/n Wi-Fi	Wi-Fi or Ethernet connection
Sensors	Not applicable	Not applicable	Wi-Fi CSI RF Antenna Interface	Not applicable
Display	Headless server	Full HD 1920x1080 monitor	Dual onboard status LEDs	Any modern display (1366x768+)

3.2.2 Software Specification

• **Server OS:** Cloud Operating System: Linux Ubuntu 22.04 LTS (Containerized via Docker on Render Cloud).

• **Firmware:** IoT Edge Firmware: ESP-IDF v5.1 / Arduino ESP32 Core in C++ for promiscuous CSI capture.

• **Backend:** Backend ASGI Engine: Python 3.10 with FastAPI 0.110 and Uvicorn 0.29.

• **Database:** Database Management System: PostgreSQL 17 hosted on Supabase Managed Cloud.

• **ORM:** ORM & Data Access: SQLModel 0.0.16 and SQLAlchemy 2.0 with connection pooling.

• **Frontend:** Frontend Framework: React 18.3 with Vite 5.0 single page application bundler.

• **Styling:** Styling & Design System: Tailwind CSS 3.4 with Google Material Symbols.

• **Testing:** Testing Tools: Selenium WebDriver 4.18, Pytest 8.1, Requests, and unittest.

• **Browser:** Client Web Browser: Google Chrome 120+, Mozilla Firefox 120+, or Microsoft Edge.

Network Requirements

• **Server Outbound:** Server Outbound Access: Outbound HTTPS/TLS connectivity to Supabase PostgreSQL (port 5432 / 6543) and external email SMTP dispatch.

• **Server Inbound:** Server Inbound Access: Public HTTPS access on port 443 routed to FastAPI ASGI service on port 8000 via Render edge proxy.

• **Client Connectivity:** Client Connectivity: Stable broadband or 4G/5G connection supporting WebSocket / Server-Sent Events (SSE) for sub-second alert streaming.

• **Security Protocols:** Security Protocols: HTTPS with TLS 1.3 encryption; secure WebSocket (WSS); Bearer token authentication.

Ports and Protocols

Table 3.5: Ports and Communication Protocols in Wi-Fi Sense

Component / Service	Port	Protocol	Purpose
FastAPI Cloud Backend	443 / 8000	HTTPS / REST	Primary API gateway handling auth, rooms, alerts, and

			telemetry
Realtime Alert Dispatcher	443 / 8000	WSS / WebSocket	Bidirectional event stream pushing instant fall alarms to nurse consoles
PostgreSQL Database	5432 / 6543	TCP (PostgreSQL)	Persistent storage for organizations, devices, alerts, and audit logs
PySerial Host Gateway	COM4 / ttyUSB0	UART @ 921600 Baud	High-speed serial transport reading raw CSI bytes from ESP32-S3
ESP32-S3 Transmitter Node	Channel 6 (2.437 GHz)	802.11 b/g/n OFDM	Periodic packet broadcasting across monitored room Fresnel zone
Vercel Frontend CDN	443	HTTPS / HTTP/2	Global edge delivery of compiled React single page application assets

Indicative Resource Allocation

- **Frontend Allocation:** Frontend Application: Minimal resource overhead; cached globally across Vercel edge CDN nodes.
- **Backend Allocation:** FastAPI Backend Service: 1-2 vCPU cores, 512 MB to 1 GB RAM allocation on Render container runtime.
- **Database Allocation:** PostgreSQL Database: 1 vCPU, 512 MB RAM on Supabase Cloud with automatic connection pooling.
- **Node Allocation:** ESP32-S3 IoT Node: 240 MHz dual-core Xtensa CPU, 512 KB SRAM, 8 MB PSRAM for CSI ring buffers.

Development Tools

- **Git:** Source Control: Git with GitHub cloud repository (Blessonjoseph07/WIFISENSE).
- **IDEs:** Integrated Development Environments: Visual Studio Code, PyCharm, and ESP-IDF extension.
- **API Tools:** API Design and Verification: FastAPI interactive Swagger UI (/docs) and Postman.
- **Database Tools:** Database Administration: Supabase Web Studio and pgAdmin 4.

3.3 SOFTWARE DESCRIPTION

This section describes the primary software technologies selected for Wi-Fi Sense and explains their architectural rationale:

3.3.1 React.js & Vite

React.js 18 is an open-source declarative JavaScript UI library. Built with Vite, it delivers a high-speed, component-based single page application. React was chosen because its virtual

DOM reconciliation efficiently renders frequently updating room status badges without reloading the entire page. Features used include functional components, custom React hooks (useAuth, useRealtimeSensing), context providers, and dynamic route rendering.

3.3.2 Python & FastAPI

Python is a versatile programming language with rich mathematical libraries (NumPy, SciPy). FastAPI provides an asynchronous ASGI framework built atop Starlette and Pydantic. It was chosen for its ultra-low latency, native async/await concurrency, declarative request validation, and automatic OpenAPI documentation. Features used include dependency injection for database sessions, JWT auth middleware, and non-blocking background tasks.

3.3.3 PostgreSQL (Supabase Managed)

PostgreSQL 17 is an enterprise-grade object-relational database. Managed via Supabase, it provides ACID transactions, strict foreign key referential integrity, and JSONB document columns for raw telemetry storage. It was chosen over NoSQL alternatives because clinical fall records and multi-tenant user permissions demand strict relational guarantees.

3.3.4 ESP-IDF & C++ Firmware

The Espressif IoT Development Framework (ESP-IDF) provides the official C/C++ development environment for ESP32-S3 chips. It provides low-level promiscuous Wi-Fi callback hooks (`esp_wifi_set_csi_rx_cb`), allowing the receiver to capture raw 64-subcarrier OFDM channel responses at hardware speed.

3.3.5 Tailwind CSS & Material Icons

Tailwind CSS is a utility-first styling framework. It enables rapid construction of sleek clinical and corporate interfaces using predetermined color tokens (slate dark modes, clinical cyan status badges, crimson emergency sirens). Google Material Symbols provide clear, standardized iconography across navigation drawers.

3.3.6 HTML5 Canvas API

The HTML5 Canvas 2D API enables direct pixel manipulation on browser viewports. Wi-Fi Sense utilizes custom canvas renderers to plot 64-subcarrier waterfall spectrogram heatmaps at 90 frames per second, bypassing DOM overhead and eliminating browser memory leaks.

3.3.7 PySerial Gateway Daemon

PySerial provides a multi-threaded Python interface to host UART serial ports. Operating at 921600 baud, it reads continuous CSI binary frames from connected receiver nodes, validates checksums, and transmits formatted JSON vectors to the FastAPI cloud backend.

3.3.8 Render Cloud & Vercel Global Edge

Render provides automated containerized deployments for the Python FastAPI ASGI service. Vercel distributes the compiled React application across global edge CDN servers with automated TLS SSL certificate provisioning, ensuring sub-50ms page load times worldwide.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

The design phase is a crucial stage in the development of Wi-Fi Sense, an AI-assisted indoor monitoring and facility intelligence platform designed to centralize contactless sensing, human activity classification, emergency fall detection, and space utilization analytics. A well-structured system design establishes the foundation for reliable operation, maintainability, strict clinical privacy, and horizontal scalability. This phase involves defining the physical RF edge architecture, database structure, graphical user interface, and integration of various micro-components.

The system design translates the requirements identified during the system study and feasibility analysis into a structured technical blueprint for implementation. It defines how the ESP32-S3 IoT transceivers capture 64-subcarrier OFDM channel states, how the host serial gateway daemon buffers and transmits telemetry, how the FastAPI ASGI backend sanitizes phase noise and evaluates activity states, and how real-time WebSocket streams push life-critical alerts to attending nurse consoles. By prioritizing usability, security, and extensibility, the architecture is designed to simplify clinical monitoring workflows while providing a solid foundation for future research enhancements.

4.2 UML DIAGRAM

Unified Modeling Language (UML) is a standard visual modeling language used to specify, visualize, design, and document the structure and behavior of software systems. It provides a common set of diagrams that help engineers and stakeholders understand system components, interactions, workflows, and deployment architecture.

In Wi-Fi Sense, nine standardized UML diagrams are used to represent the system architecture, user interactions, internal components, and operational workflows. These diagrams provide a clear understanding of how clinical caregivers, facility managers, corporate staff, and family members interact with the platform, and how its distributed components communicate to support real-time contactless sensing and emergency dispatch.

4.2.1 Use Case Diagram

A use case diagram is a graphical representation that illustrates how users and external entities interact with a system. It helps identify, outline, and organize the system's functional requirements from an actor's perspective. Built using UML standards, use case diagrams define system boundaries, actors, use cases, and relationships.

Figure 4.1 shows the use cases of Wi-Fi Sense across its administrative and clinical access levels. Primary actors include the System Administrator, Organization Administrator, Facility Manager, Caregiver/Staff, Corporate Facility Staff, Family Member/Doctor, and the physical ESP32-S3 CSI Sensing Node.

- **Administration:** Administration: Manage Organizations, Manage Users & Roles, Provision Facilities.
- **Facility Management:** Facility Management: Manage Buildings, Floors, Rooms, and Sensing Devices.
- **Sensing & AI:** Sensing & AI: Collect CSI Data, Preprocess CSI Data, Detect Presence, Recognize Activity, Detect Potential Fall.
- **Monitoring:** Monitoring: View Central Monitoring Dashboard, View Room Occupancy, Monitor Resident Presence, Monitor Activity.
- **Alert Management:** Alert Management: Generate Alert, Send Notification, View Alert History, Acknowledge Alert, Resolve Alert.
- **Analytics:** Analytics: View Occupancy Analytics, View Room Utilization, View Historical Activity.

Figure 4.1: Use Case Diagram for Wi-Fi Sense Ecosystem

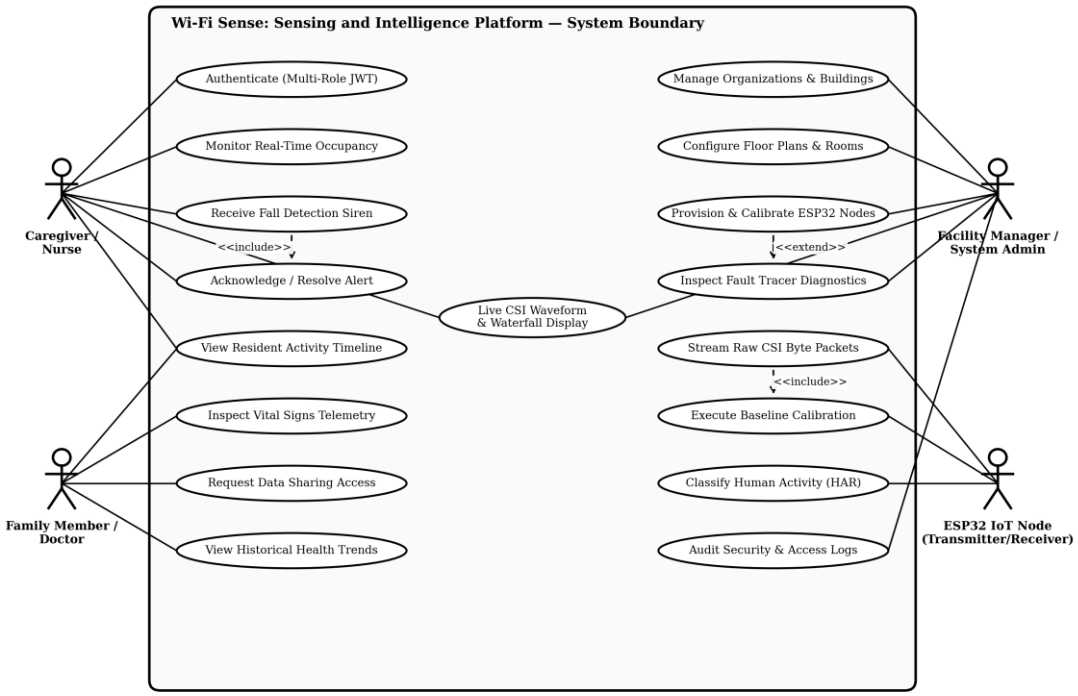


Figure 4.1: Use case diagram of Wi-Fi Sense

4.2.2 Sequence Diagram

A sequence diagram is a type of interaction diagram that depicts the chronological flow of messages between system components. It helps visualize object interactions, understand system requirements, and detect potential concurrency bottlenecks. Key elements include actors, lifelines, messages, and execution activations.

Figure 4.2 shows the end-to-end emergency fall detection and dispatch interaction sequence. When a resident falls, the ESP32-S3 node captures raw CSI data and transmits it to the CSI Processing Service. The processed features are passed to the ML Classification Service, which identifies a high-confidence fall. The backend creates an Alert record, and the Alert Engine broadcasts an instant notification to the Caregiver Console. The caregiver acknowledges the alert, investigates the room, and records the resolution.

Figure 4.2: Sequence Diagram for CSI Packet Ingestion, Inference, and Fall Alert Dispatch

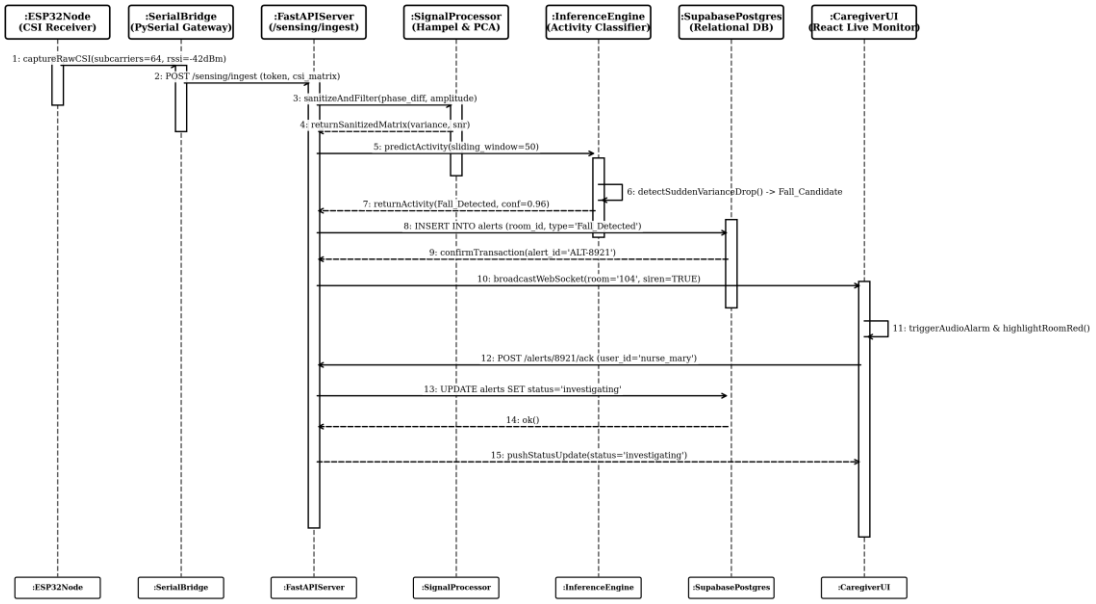


Figure 4.2: Sequence diagram for fall detection and caregiver dispatch in Wi-Fi Sense

4.2.3 State Chart Diagram

A State Chart Diagram is a UML behavioral diagram that represents the various states an entity can exist in and how it transitions between them in response to internal and external events. Key components include the Initial State, States, Transitions, Events, Actions, and Final States.

Figure 4.3 shows the lifecycle of an emergency alert record in Wi-Fi Sense. The states follow the alert status field: Detection, Created, Notified, Acknowledged, and Resolved. Transitions between states require explicit caregiver actions and log responder identity for clinical accountability.

Figure 4.3: State Machine Diagram for Room Sensing and Emergency Incident Lifecycle

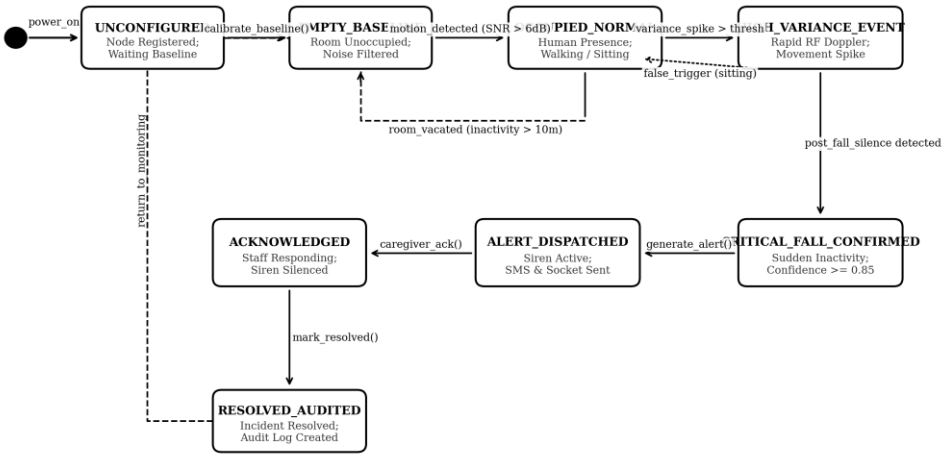


Figure 4.3: State chart diagram of an emergency alert record

4.2.4 Activity Diagram

An activity diagram illustrates the procedural flow of control from activity to activity within a system workflow. It models sequential steps, decision branching points, and concurrent parallel activities using standardized symbols:

- **Initial Node:** Initial node: Black circle representing workflow starting point.
- **Activity:** Activity: Rounded rectangle containing the operational task.
- **Decision Node:** Decision node: Diamond shape representing conditional branching.
- **Final Node:** Final node: Encircled black circle representing workflow completion.

Figure 4.4 shows the activity workflow for continuous CSI packet processing, feature extraction, activity classification, and emergency escalation.

Figure 4.4: Activity Diagram for Ingestion Pipeline and Fall Inference Workflow

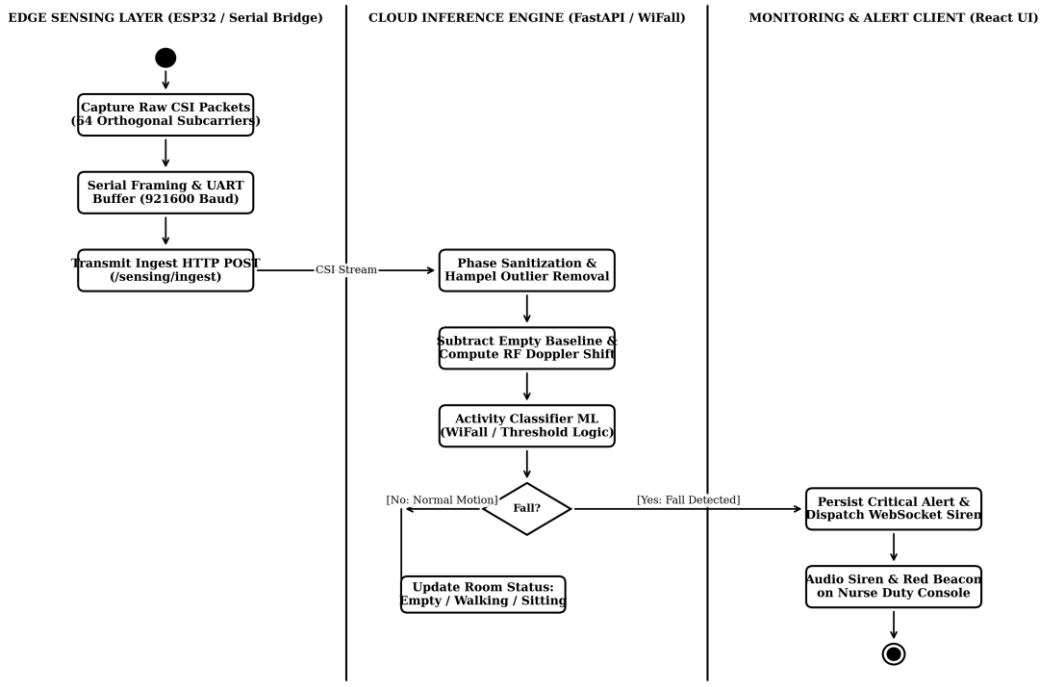


Figure 4.4: Activity diagram for CSI ingestion and fall incident workflow

4.2.5 Class Diagram

A Class Diagram is a key UML component used in object-oriented modeling to represent the static structure of an application. It defines classes, attributes, methods, and relationships such as association, aggregation, composition, and inheritance.

Figure 4.5 shows the persistent domain classes of Wi-Fi Sense matching the relational database schema: Organization, Building, Floor, Room, User, Role, SensingDevice, SensingEvent, ActivityType, Alert, and Notification.

Figure 4.5: Class Diagram for System Relational Architecture and Sensing Models

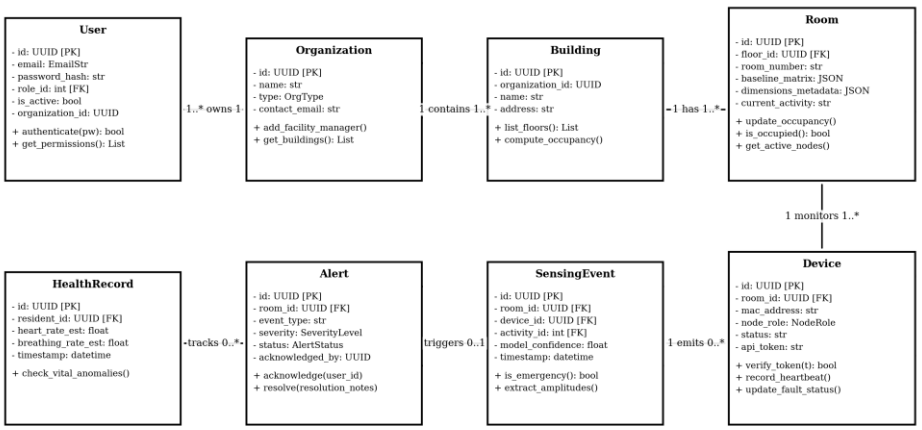


Figure 4.5: Class diagram of Wi-Fi Sense domain entities and relations

4.2.6 Object Diagram

Object diagrams represent a snapshot of the system's runtime state at a particular point in time, showing concrete instances of classes and their attribute values.

Figure 4.6 shows an object diagram with instantiated entities during an active fall event: Organization (St. Peter's Elder Care Home), Room (Room 204), Device (ESP32-S3 Node), Resident (Annamma Joseph), Alert (Fall_Detected, Severity CRITICAL), and Caregiver (Sr. Mary Sebastian).

Figure 4.6: Object Diagram for Active Room Telemetry and Fall Incident Snapshot

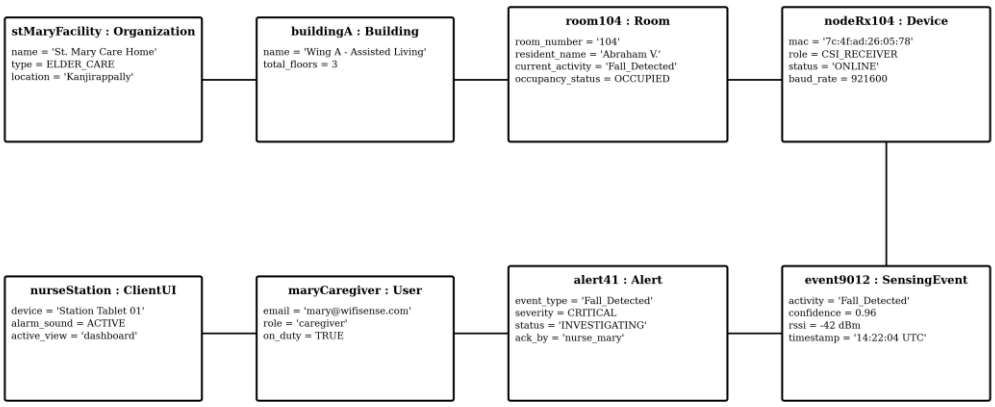


Figure 4.6: Object diagram showing runtime instance state during active fall event

4.2.7 Component Diagram

A component diagram illustrates how software components are organized and wired together to form larger subsystems. Figure 4.7 illustrates the modular subsystems of Wi-Fi Sense: the React Web Client, FastAPI API Gateway, Authentication Service, CSI Signal Processing Service, WiFall Inference Engine, Alert Management Service, and Supabase PostgreSQL Database.

Figure 4.7: Component Diagram for System Architecture and Interface Contracts

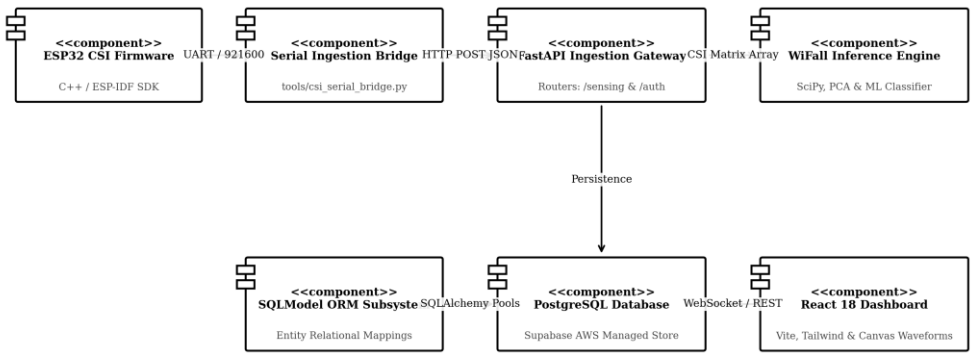


Figure 4.7: Component diagram of Wi-Fi Sense software architecture

4.2.8 Deployment Diagram

A deployment diagram models the physical hardware nodes and runtime environments on which the software artifacts execute. Figure 4.8 shows the distributed deployment topology: the ESP32-S3 IoT sensing hardware in monitored rooms, the Host Gateway PC connected via USB COM port, the Render Linux Cloud container running the FastAPI backend, the Supabase Cloud PostgreSQL database, and client web browsers connected over secure HTTPS and WSS.

Figure 4.8: Deployment Diagram for Edge Hardware, Render Backend, and Supabase Cloud

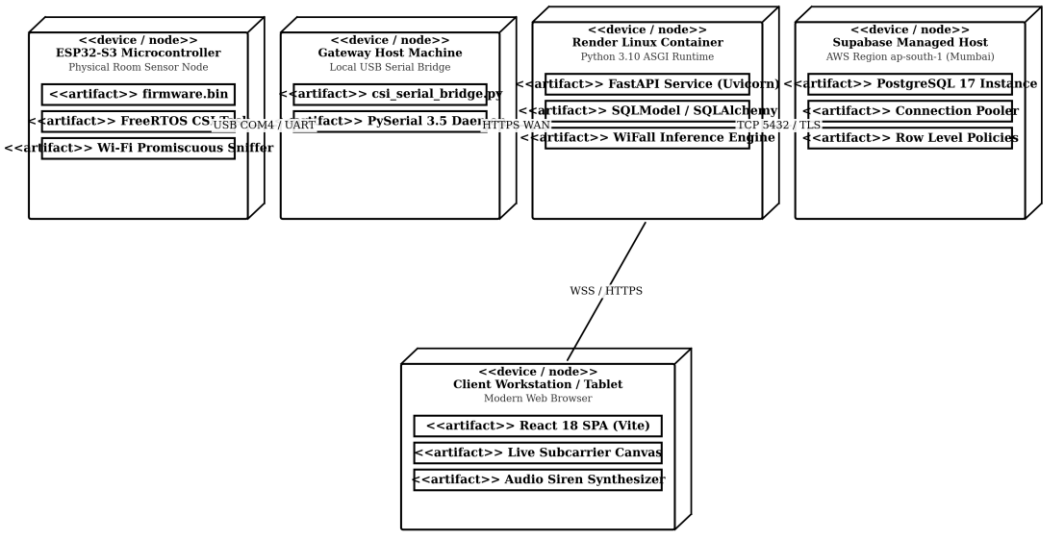


Figure 4.8: Deployment diagram of Wi-Fi Sense hardware and cloud topology

4.2.9 Collaboration Diagram

A collaboration (communication) diagram shows the structural relationships between cooperating objects, emphasizing the message links connecting them. Figure 4.9 illustrates the collaboration between the User, React Client, FastAPI Gateway, PostgreSQL Database, and WebSocket Alert Dispatcher during authentication and incident triage.

Figure 4.9: Collaboration Diagram for Emergency Fall Detection & Response Protocol

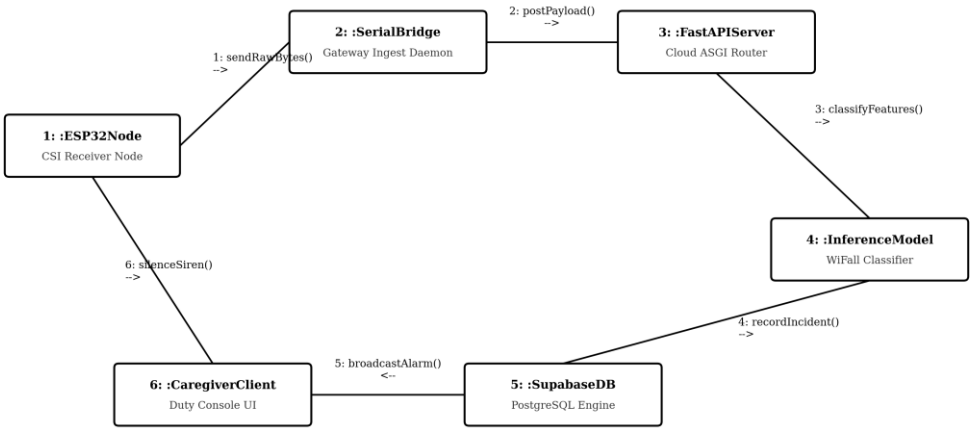


Figure 4.9: Collaboration diagram for emergency fall detection and response

4.3 USER INTERFACE DESIGN USING FIGMA

This section presents the user interface design screens and live implemented interfaces of Wi-Fi Sense. The interface is architected around role-governed application shells: CareAppShell for clinical staff, SpaceAppShell for corporate workplace managers, FamilyPortalShell for remote relatives, and SystemAdminShell for administrators. The interface emphasizes high visual clarity, glanceable room status matrices, and non-blocking alert banners.

Form Name: Landing Page

The landing page introduces the platform, providing an interactive 3D Wi-Fi CSI signal propagation wireframe globe, channel state telemetry cards (Signal RSSI, Variance, CSI Frame counter), and direct links to role-based authentication. See Figure 4.10.

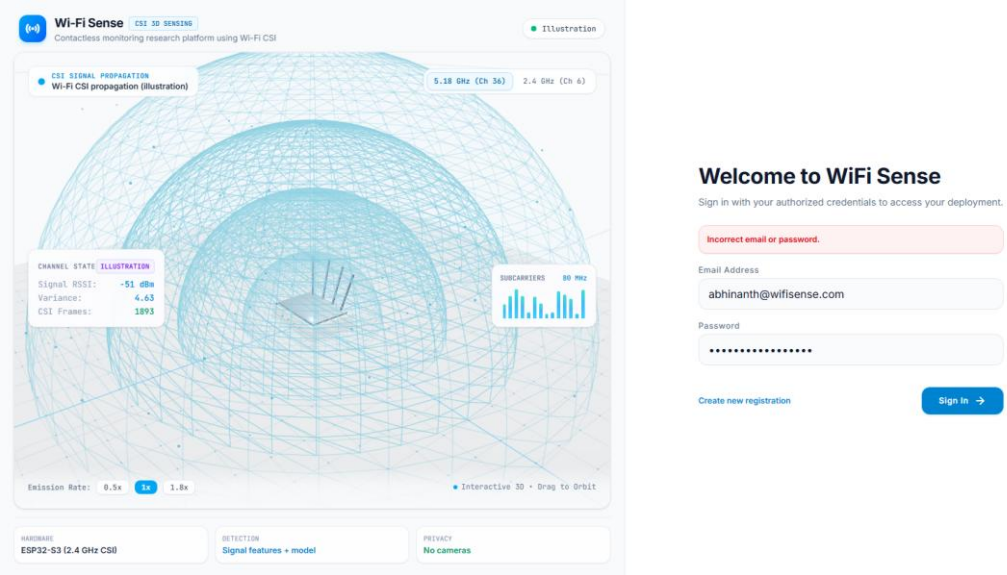


Figure 4.10: Landing page and 3D CSI signal visualizer of Wi-Fi Sense

Form Name: Sign In

The sign-in page offers secure credential authentication, role-based contextual routing, password masking, and quick-fill demo accounts for System Admin, Caregiver, Facility Manager, and Corporate Staff. See Figure 4.11.

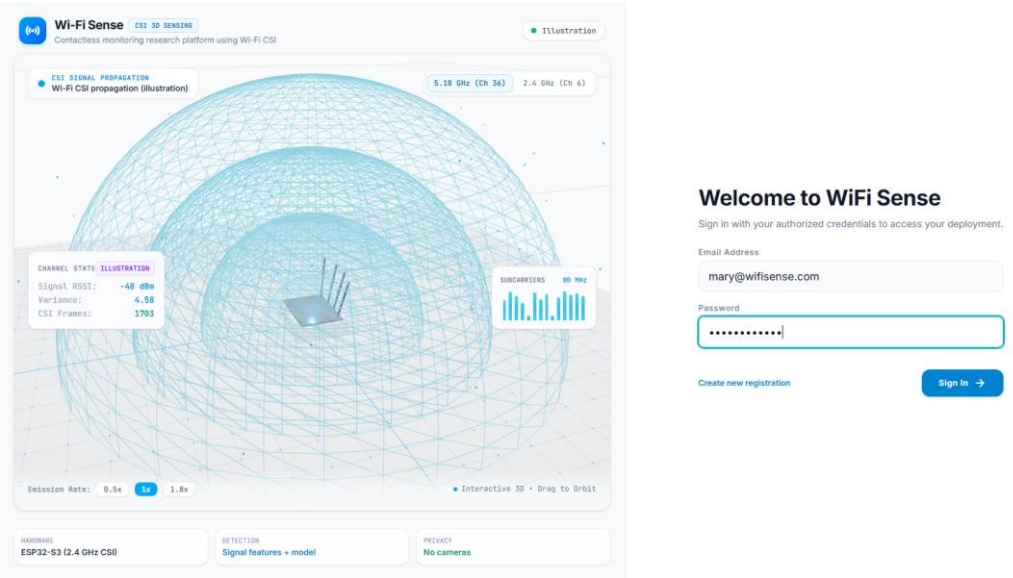


Figure 4.11: Sign-in and authentication page

Form Name: Central Monitoring Dashboard

The Central Monitoring dashboard displays real-time room occupancy cards across wings, active fall risk badges, subcarrier amplitude spectrogram streams, and a prominent emergency fall alert banner with one-tap Acknowledge and Resolve actions. See Figure 4.12.

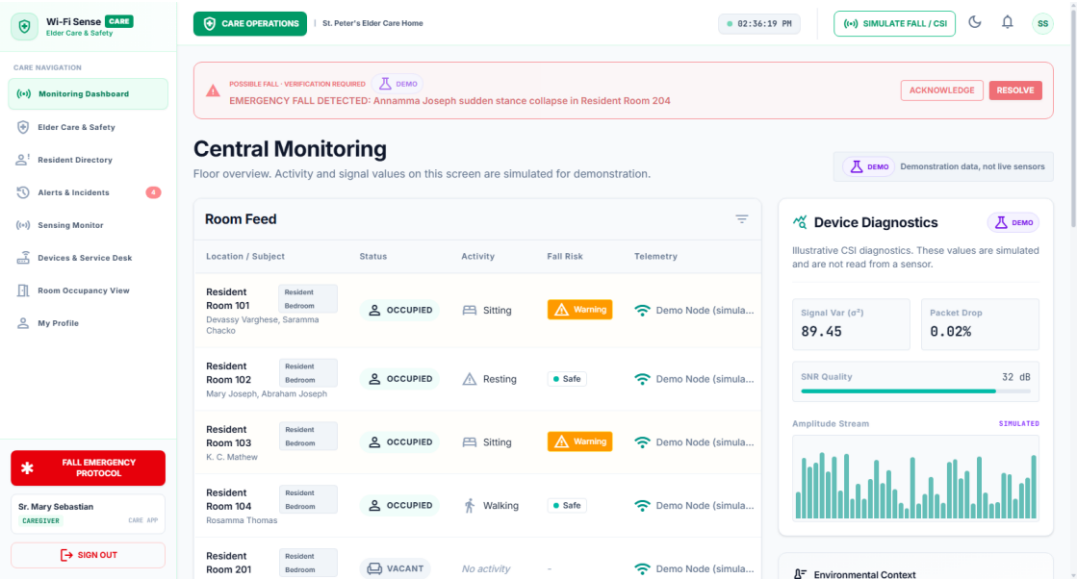


Figure 4.12: Central Monitoring dashboard with active emergency fall banner

Form Name: Family Data Sharing Policies

The Family Data Sharing Policies page allows facility managers to configure zero-leak privacy policies, toggling room presence disclosures, activity details, room suite identifiers, and alert severity thresholds for external family members. See Figure 4.13.

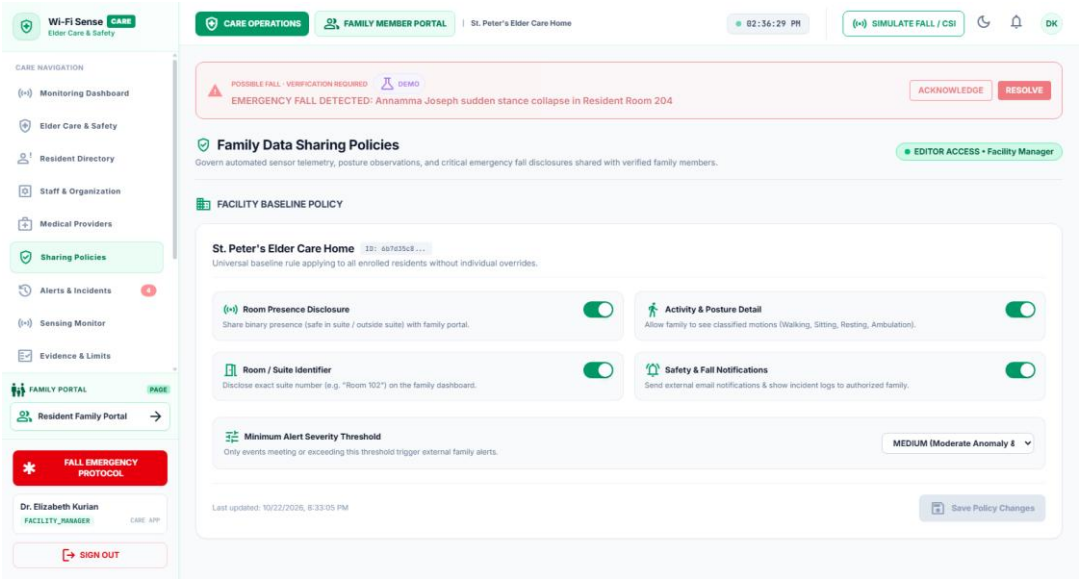


Figure 4.13: Family data sharing and privacy policy configuration

Form Name: Family Care Portal

The Family Care Portal allows approved family sponsors to verify resident presence, review safety incident logs, and manage monthly Care Plan subscriptions without receiving intrusive visual imagery. See Figure 4.14.

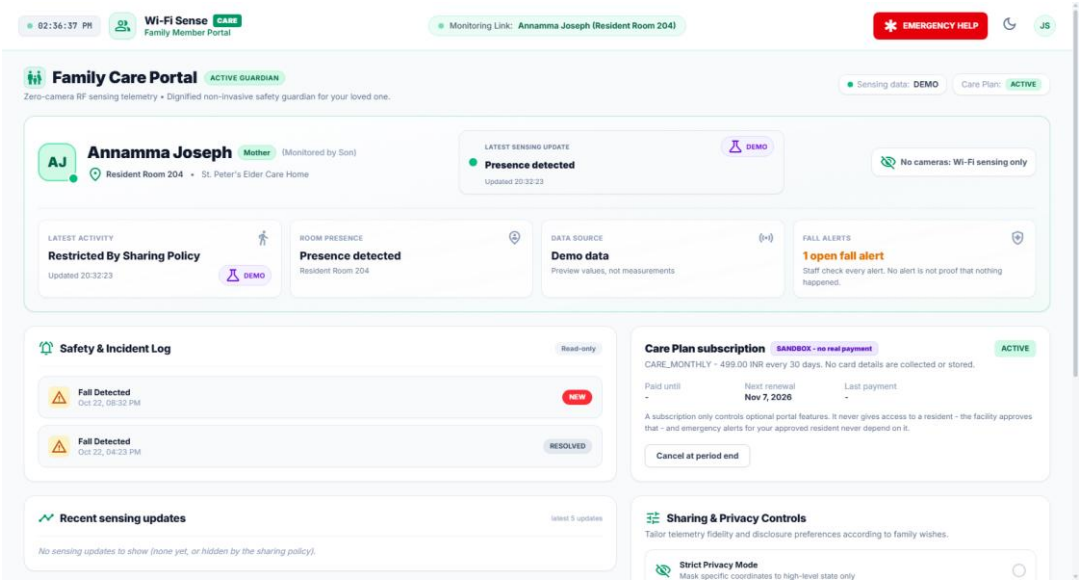


Figure 4.14: Family Care Portal and resident wellness view

Form Name: Corporate Space & Energy Intelligence

The Corporate Space & Energy Intelligence dashboard renders meeting room occupancy heatmaps, calculates peak utilization rates, and highlights vacant rooms wasting HVAC power, providing one-click energy mitigation controls. See Figure 4.15.

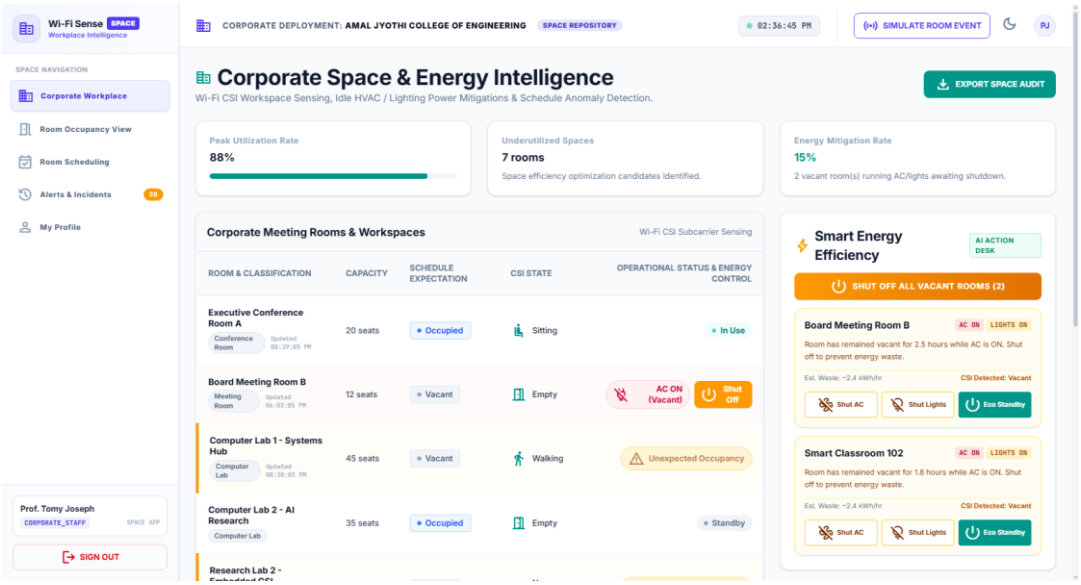


Figure 4.15: Corporate workplace space and energy intelligence dashboard

Form Name: Elder Care Sensing Nodes & Service Desk

The Sensing Nodes page displays the hardware sensor fleet, ESP32 MAC addresses, authentication tokens, assigned room locations, firmware versions, and hardware fault tracer tickets. See Figure 4.16.

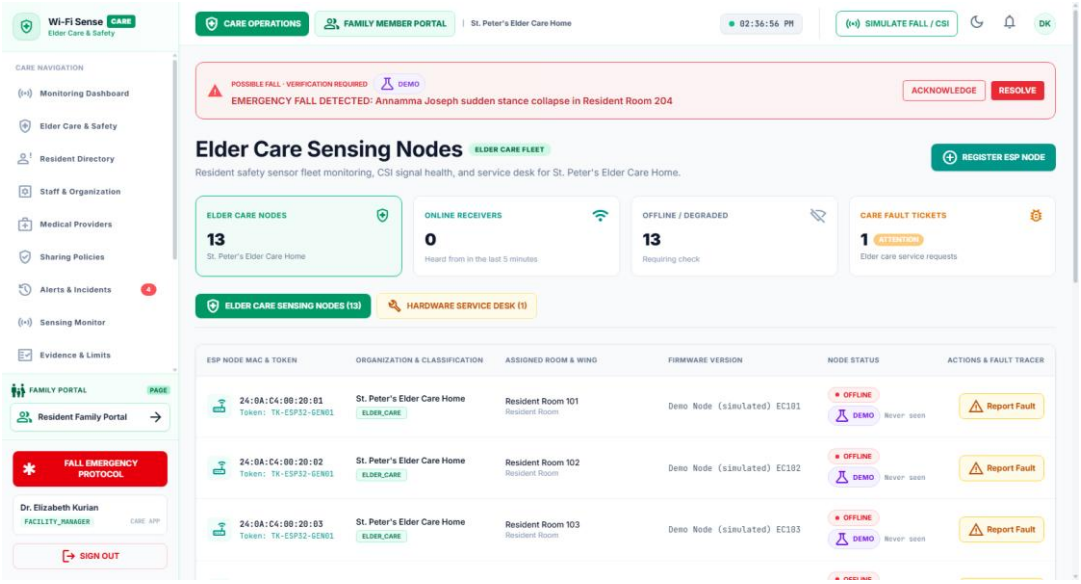


Figure 4.16: Elder Care sensing nodes and hardware service desk

Form Name: Resident Accounts Directory

The Resident Accounts page manages resident profiles, assigned room numbers, emergency contact records, and one-touch emergency inspection actions. See Figure 4.17.

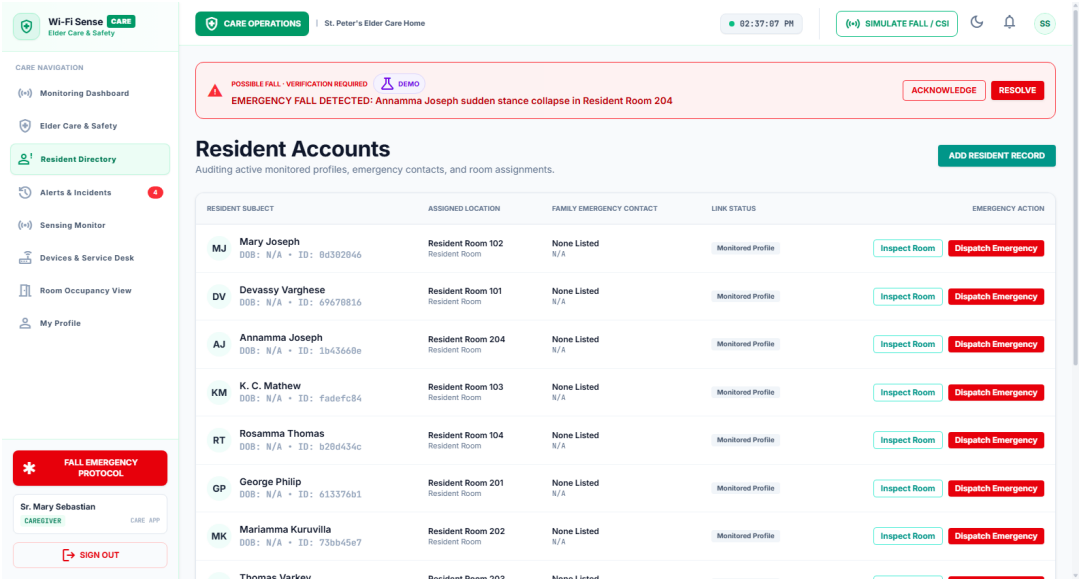


Figure 4.17: Resident accounts directory and room assignments

Form Name: Multi-Tenant System Administration Console

The System Administration console provides global tenant oversight across elder care and corporate organizations, system metrics, telemetry simulation controls, and administrative audit logging. See Figure 4.18.

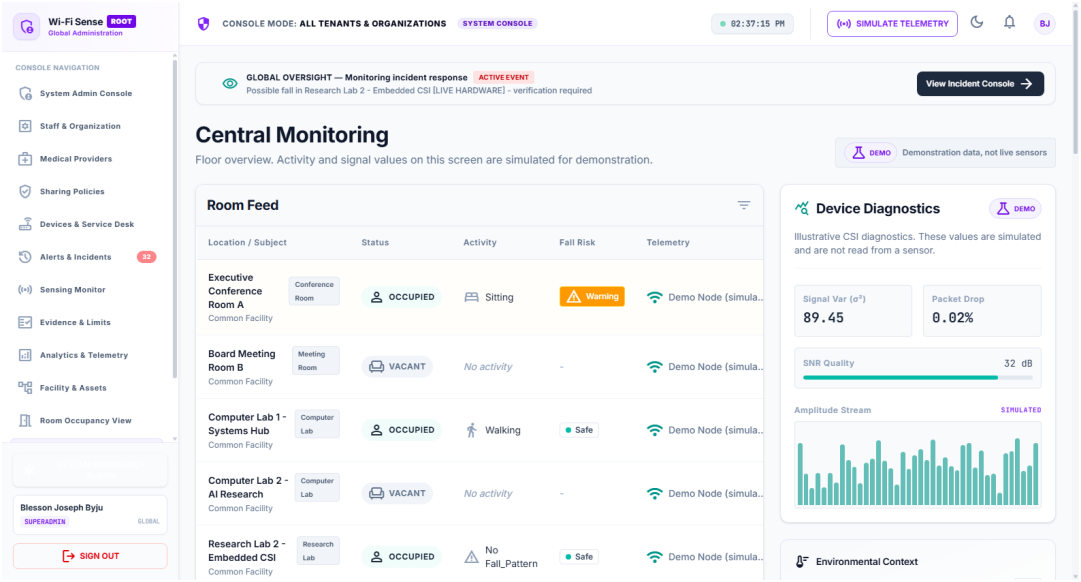


Figure 4.18: Multi-tenant system administration console

Application Pages Declared in Routing Module

Table 4.1: Application Pages Declared in Frontend Routing Architecture

Page Component	Route Path	Allowed Roles	Purpose
LandingSplash	/	Public	3D CSI wireframe visualization and platform introduction
AuthPage / Login	/login	Public	User authentication, JWT issuance, and demo role

			shortcuts
CareAppShell	/dashboard (care)	Caregiver, Facility Manager	Clinical nursing console with live room feed and siren alerts
SpaceAppShell	/dashboard (space)	Corporate Staff, Facility Admin	Workplace occupancy matrix and HVAC power mitigation
FamilyPortalShell	/family	Family Member / Guardian	Non-visual resident wellness check, safety history, Care Plan
DoctorPortalShell	/doctor	Doctor / Physician	Clinical resident review, vital sign telemetry, doctor notes
SystemAdminShell	/admin	System Administrator	Global multi-tenant governance, organization setup, audit logs
DevicesView	/devices	Facility Manager, Admin	ESP32-S3 sensor fleet inventory, MAC tokens, fault ticketing
ResidentsView	/residents	Caregiver, Facility Manager	Resident account directory, room assignments, emergency links
SharingPolicyManager	/policies	Facility Manager	Zero-leak family telemetry disclosure and severity filters
AlertsView	/alerts	Caregiver, Manager, Admin	Emergency incident history, resolution notes, and audit trail

4.4 DATABASE DESIGN

A database is an organized collection of structured data designed to support efficient storage, retrieval, and relational integrity. In Wi-Fi Sense, the persistence layer manages organizations, buildings, rooms, sensing nodes, high-frequency CSI telemetry events, activity classifications, and emergency alert logs.

4.4.1 Relational Database Management System (RDBMS)

PostgreSQL serves as the RDBMS for Wi-Fi Sense. The core relational concepts implemented include:

- **Primary Key:** Primary Key: Uniquely identifies each tuple in an entity table (UUID, SERIAL, or BIGSERIAL).
- **Foreign Key:** Foreign Key: Enforces referential integrity by linking child entity records to parent tables.
- **Entity Integrity:** Entity Integrity: Guarantees that primary key values are non-null and uniquely distinct.
- **Referential Integrity:** Referential Integrity: Ensures that foreign keys reference valid parent rows with cascading deletes where appropriate.
- **Domain Constraints:** Domain Constraints: Restricts column values to valid enumerations (ORGANIZATION_TYPE, ALERT_STATUS_TYPE).
- **Candidate Key:** Candidate Key: Minimal superkey candidate capable of uniquely distinguishing rows.
- **Super Key:** Super Key: Any set of attributes uniquely identifying records within a relational table.

4.4.2 Normalization

Normalization is the systematic process of organizing database relations to minimize data redundancy and prevent insertion, update, and deletion anomalies. The design of Wi-Fi Sense enforces normalization up to Third Normal Form (3NF). Below is an illustrative walkthrough of the normalization stages:

First Normal Form (1NF)

A table is in First Normal Form when each column contains atomic values, each record is uniquely identifiable, and there are no repeating groups or comma-separated lists.

Example (Before 1NF — Unnormalized Table):

Table: Unnormalized Example Relation (Before 1NF)

Student_ID	Student_Name	Courses	Contact

101	Alice	Math, Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science, History	7654321098

Conversion to 1NF (Atomic Attributes):

Table: Relation Converted to First Normal Form (1NF)

Student_ID	Student_Name	Course	Contact
101	Alice	Math	9876543210
101	Alice	Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science	7654321098
103	Charlie	History	7654321098

Second Normal Form (2NF)

A relation is in Second Normal Form (2NF) if it is in 1NF and all non-key attributes are fully functionally dependent on the complete primary key, removing partial dependencies on composite keys.

Conversion to 2NF (Decomposed into Students and Enrollment Tables):

Table: Students Relation (2NF)

Student_ID	Student_Name	Contact
101	Alice	9876543210
102	Bob	8765432109
103	Charlie	7654321098

Table: Course Enrollment Relation (2NF)

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and there are no transitive dependencies: non-key attributes must not depend on other non-key attributes. In Wi-Fi Sense, rooms reference floor IDs, floors reference building IDs, and buildings reference organization IDs, ensuring zero transitive updates.

4.4.3 Sanitization

Data sanitization and input validation techniques protect Wi-Fi Sense against injection attacks and malformed telemetry:

- **Validation:** Input Validation: Pydantic schemas enforce strict type validation, field length bounds, and regex formats.
- **XSS Defense:** Output Encoding: React automatically encodes dynamic variables within JSX to prevent Cross-Site Scripting (XSS).
- **SQLi Defense:** SQL Injection Prevention: SQLModel/SQLAlchemy executes parameterized queries exclusively, avoiding string concatenation.
- **API Security:** API Validation: Ingestion endpoints validate MAC addresses and node tokens prior to parsing CSI payloads.
- **Credential Protection:** Credential Protection: Passwords are salted and hashed using Argon2/BCrypt; secrets are never logged in responses.

4.4.4 Indexing

Database indexing significantly accelerates query performance for real-time room monitoring and incident lookups. PostgreSQL automatically builds B-tree indexes on primary keys and unique constraints. In Wi-Fi Sense, additional indexes are established on foreign keys and high-frequency query columns:

```
CREATE INDEX idx_sensing_events_room_timestamp ON sensing_events (room_id, timestamp
DESC);
CREATE INDEX idx_alerts_room_status ON alerts (room_id, status);
CREATE INDEX idx_devices_mac ON devices (mac_address);
```

4.5 TABLE DESIGN

The database is PostgreSQL hosted on Supabase Cloud. The tables below are listed as designed and implemented in the Wi-Fi Sense relational schema, fully normalized to 3NF:

1. ORGANIZATIONS

Primary key: organization_id

Foreign key: Nil

Table 4.2: ORGANIZATIONS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	organization_id	UUID	PK	Unique organization identifier
2	name	VARCHAR(150)	UNIQUE, NOT NULL	Organization name
3	type	VARCHAR(20)	NOT NULL	Elder Care or Corporate
4	created_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Creation date
5	updated_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Last updated date

2. BUILDINGS

Primary key: building_id

Foreign key: organization_id references ORGANIZATIONS

Table 4.3: BUILDINGS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	building_id	UUID	PK	Unique building identifier
2	organization_id	UUID	FK	References ORGANIZATIONS
3	name	VARCHAR(100)	NOT NULL	Building name
4	address	TEXT	NULL	Building address
5	created_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Creation date

3. FLOORS

Primary key: floor_id

Foreign key: building_id references BUILDINGS

Table 4.4: FLOORS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	floor_id	UUID	PK	Unique floor identifier
2	building_id	UUID	FK	References BUILDINGS

3	floor_number	INTEGER	NOT NULL	Floor number
4	floor_plan_url	VARCHAR(255)	NULL	Optional floor plan reference

4. ROOMS

Primary key: room_id

Foreign key: floor_id references FLOORS

Table 4.5: ROOMS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	room_id	UUID	PK	Unique room identifier
2	floor_id	UUID	FK	References FLOORS
3	name	VARCHAR(50)	NOT NULL	Room name
4	room_type	VARCHAR(50)	NULL	Type of room
5	capacity	INTEGER	NULL	Maximum room capacity
6	dimensions_metadata	JSONB	NULL	Room dimension/layout information

5. USERS

Primary key: user_id

Foreign key: Nil

Table 4.6: USERS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	user_id	UUID	PK	Unique user identifier
2	email	VARCHAR(150)	UNIQUE, NOT NULL	User email
3	password_hash	VARCHAR(255)	NOT NULL	Securely hashed password
4	first_name	VARCHAR(50)	NOT NULL	First name
5	last_name	VARCHAR(50)	NULL	Last name
6	is_active	BOOLEAN	DEFAULT TRUE	Account status
7	created_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Registration date

6. ROLES

Primary key: role_id

Foreign key: Nil

Table 4.7: ROLES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	role_id	INTEGER	PK	Unique role identifier

2	name	VARCHAR(80)	UNIQUE, NOT NULL	System role name

7. USER_ROLES

Primary key: user_role_id

Foreign keys: user_id references USERS; role_id references ROLES; organization_id references ORGANIZATIONS; building_id references BUILDINGS; room_id references ROOMS

Table 4.8: USER_ROLES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	user_role_id	UUID	PK	Unique mapping identifier
2	user_id	UUID	FK	References USERS
3	role_id	INTEGER	FK	References ROLES
4	organization_id	UUID	FK, NULL	Organization-level access
5	building_id	UUID	FK, NULL	Building-level access
6	room_id	UUID	FK, NULL	Room-level access

8. DEVICES

Primary key: device_id

Foreign key: room_id references ROOMS

Table 4.9: DEVICES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	device_id	UUID	PK	Unique sensing-device identifier
2	room_id	UUID	FK, NULL	References ROOMS
3	mac_address	VARCHAR(17)	UNIQUE, NOT NULL	ESP32-S3 MAC address
4	device_status	VARCHAR(20)	NOT NULL	Online, Offline or Maintenance
5	firmware_version	VARCHAR(30)	NULL	Firmware version
6	last_seen_at	TIMESTAMPTZ	NULL	Last communication time

9. ACTIVITY_TYPES

Primary key: activity_type_id

Foreign key: Nil

Table 4.10: ACTIVITY_TYPES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	activity_type_id	INTEGER	PK	Unique activity identifier

2	name	VARCHAR(50)	UNIQUE, NOT NULL	Empty, Presence, Walking, Sitting, Fall_Detected
3	category	VARCHAR(20)	NOT NULL	STATUS, ACTIVITY or CRITICAL

10. SENSING_TEMPLATES

Primary key: template_id

Foreign keys: organization_id references ORGANIZATIONS; activity_type_id references ACTIVITY_TYPES

Table 4.11: SENSING_TEMPLATES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	template_id	UUID	PK	Unique template identifier
2	organization_id	UUID	FK, NULL	References ORGANIZATIONS
3	activity_type_id	INTEGER	FK	References ACTIVITY_TYPES
4	template_name	VARCHAR(100)	NOT NULL	CSI template name
5	csi_amplitude_baseline	DOUBLE PRECISION[]	NULL	Reference CSI amplitude profile
6	csi_phase_baseline	DOUBLE PRECISION[]	NULL	Reference CSI phase profile
7	environment_metadata	JSONB	NULL	Room/environment information
8	created_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Creation date
9	updated_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Last updated date

11. SENSING_EVENTS

Primary key: event_id

Foreign keys: device_id references DEVICES; room_id references ROOMS; inferred_activity_id references ACTIVITY_TYPES

Table 4.12: SENSING_EVENTS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	event_id	BIGSERIAL	PK	Unique sensing event
2	device_id	UUID	FK	References DEVICES
3	room_id	UUID	FK	References ROOMS
4	timestamp	TIMESTAMPTZ	NOT NULL	Sensing timestamp

5	rssi	INTEGER	NULL	Received Signal Strength
6	subcarrier_count	INTEGER	NULL	Number of CSI subcarriers
7	raw_csi_payload_path	VARCHAR(512)	NULL	Storage path for raw CSI
8	extracted_features	JSONB	NULL	Preprocessed CSI features
9	inferred_activity_id	INTEGER	FK, NULL	References ACTIVITY_TYPES
10	model_confidence	NUMERIC(5,4)	NULL	ML prediction confidence

12. ALERT_TEMPLATES

Primary key: alert_template_id

Foreign keys: organization_id references ORGANIZATIONS; event_type_id references ACTIVITY_TYPES

Table 4.13: ALERT_TEMPLATES Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	alert_template_id	UUID	PK	Unique alert template
2	organization_id	UUID	FK, NULL	References ORGANIZATIONS
3	event_type_id	INTEGER	FK	References ACTIVITY_TYPES
4	alert_severity	VARCHAR(20)	NOT NULL	LOW, MEDIUM, HIGH or CRITICAL
5	message_template	TEXT	NOT NULL	Alert message format
6	cooldown_period_seconds	INTEGER	DEFAULT 0	Duplicate-alert prevention period

13. ALERT_CONFIGURATIONS

Primary key: alert_configuration_id

Foreign keys: room_id references ROOMS; alert_template_id references ALERT_TEMPLATES

Table 4.14: ALERT_CONFIGURATIONS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	alert_configuration_id	UUID	PK	Unique configuration
2	room_id	UUID	FK	References ROOMS
3	alert_template_id	UUID	FK	References ALERT_TEMPLATES
4	is_enabled	BOOLEAN	DEFAULT TRUE	Configuration status

14. ALERTS

Primary key: alert_id

Foreign keys: alert_configuration_id references ALERT_CONFIGURATIONS; room_id references ROOMS

Table 4.15: ALERTS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	alert_id	UUID	PK	Unique alert identifier
2	alert_configuration_id	UUID	FK, NULL	References ALERT_CONFIGURATIONS
3	room_id	UUID	FK	References ROOMS
4	event_type	VARCHAR(50)	NOT NULL	Event that triggered alert
5	severity	VARCHAR(20)	NOT NULL	Alert severity
6	message	TEXT	NOT NULL	Generated alert message
7	status	VARCHAR(20)	NOT NULL	DETECTION, CREATED, NOTIFIED, ACKNOWLEDGED or RESOLVED
8	created_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Alert creation time

15. ALERT_ACKNOWLEDGEMENTS

Primary key: acknowledgement_id

Foreign keys: alert_id references ALERTS; user_id references USERS

Table 4.16: ALERT_ACKNOWLEDGEMENTS Schema

No.	Field name	Datatype (Size)	Key Constraints	Description of the field
1	acknowledgement_id	UUID	PK	Unique acknowledgement identifier
2	alert_id	UUID	FK	References ALERTS
3	user_id	UUID	FK, NULL	User who acknowledged/resolved
4	acknowledged_at	TIMESTAMPTZ	DEFAULT CURRENT_TIMESTAMP	Acknowledgement time
5	resolved_at	TIMESTAMPTZ	NULL	Resolution time
6	resolution_notes	TEXT	NULL	Resolution details

PRIMARY KEY SUMMARY

Table 4.17: Primary Key Summary Across All Relational Entities

Table	Primary Key
ORGANIZATIONS	organization_id
BUILDINGS	building_id
FLOORS	floor_id
ROOMS	room_id
USERS	user_id
ROLES	role_id
USER_ROLES	user_role_id
DEVICES	device_id
ACTIVITY_TYPES	activity_type_id
SENSING_TEMPLATES	template_id
SENSING_EVENTS	event_id
ALERT_TEMPLATES	alert_template_id
ALERT_CONFIGURATIONS	alert_configuration_id
ALERTS	alert_id
ALERT_ACKNOWLEDGEMENTS	acknowledgement_id

FOREIGN KEY SUMMARY

Table 4.18: Foreign Key Summary and Referential Target Mappings

Table	Foreign Key	References
BUILDINGS	organization_id	ORGANIZATIONS
FLOORS	building_id	BUILDINGS
ROOMS	floor_id	FLOORS
USER_ROLES	user_id	USERS
USER_ROLES	role_id	ROLES
USER_ROLES	organization_id	ORGANIZATIONS
USER_ROLES	building_id	BUILDINGS
USER_ROLES	room_id	ROOMS
DEVICES	room_id	ROOMS
SENSING_TEMPLATES	organization_id	ORGANIZATIONS
SENSING_TEMPLATES	activity_type_id	ACTIVITY_TYPES
SENSING_EVENTS	device_id	DEVICES
SENSING_EVENTS	room_id	ROOMS
SENSING_EVENTS	inferred_activity_id	ACTIVITY_TYPES
ALERT_TEMPLATES	organization_id	ORGANIZATIONS
ALERT_TEMPLATES	event_type_id	ACTIVITY_TYPES
ALERT_CONFIGURATIONS	room_id	ROOMS
ALERT_CONFIGURATIONS	alert_template_id	ALERT_TEMPLATES
ALERTS	alert_configuration_id	ALERT_CONFIGURATIONS

ALERTS	room_id	ROOMS
ALERT_ACKNOWLEDGEMENTS	alert_id	ALERTS
ALERT_ACKNOWLEDGEMENTS	user_id	USERS

TABLE RELATIONSHIPS

Table 4.19: Cardinality and Multiplicity Relationships

Parent Table	Child Table	Relationship
ORGANIZATIONS	BUILDINGS	One-to-Many
BUILDINGS	FLOORS	One-to-Many
FLOORS	ROOMS	One-to-Many
ROOMS	DEVICES	One-to-Many
ROOMS	SENSING_EVENTS	One-to-Many
DEVICES	SENSING_EVENTS	One-to-Many
ACTIVITY_TYPES	SENSING_EVENTS	One-to-Many
ORGANIZATIONS	SENSING_TEMPLATES	One-to-Many
ACTIVITY_TYPES	SENSING_TEMPLATES	One-to-Many
USERS	USER_ROLES	One-to-Many
ROLES	USER_ROLES	One-to-Many
ROOMS	ALERT_CONFIGURATIONS	One-to-Many
ALERT_TEMPLATES	ALERT_CONFIGURATIONS	One-to-Many
ALERT_CONFIGURATIONS	ALERTS	One-to-Many
ROOMS	ALERTS	One-to-Many
ALERTS	ALERT_ACKNOWLEDGEMENTS	One-to-Many
USERS	ALERT_ACKNOWLEDGEMENTS	One-to-Many

Database Security

- **Password Hashing:** User passwords are encrypted using secure salted hashing (Argon2 / BCrypt) rather than plain text.
- **RBAC Scoping:** Role-Based Access Control (RBAC) restricts database read/write queries according to assigned user scopes.
- **Tenant Isolation:** Organization, building, and room-level scopes restrict clinical monitoring and telemetry access.
- **Referential Integrity:** Foreign-key constraints with ON DELETE CASCADE and RESTRICT maintain absolute referential integrity.
- **SQL Injection Defense:** Input validation and parameterized ORM queries eliminate SQL injection vulnerabilities.
- **Data Privacy:** Sensitive raw CSI features and health records are restricted to authenticated clinical personnel.
- **Auditability:** Alert acknowledgement and resolution audit logs record timestamps and

responder user IDs for accountability.

Database Module Flow

The relational data processing pipeline follows a deterministic operational progression:

ORGANIZATION --> BUILDING --> FLOOR --> ROOM --> DEVICE --> SENSING EVENT --> ACTIVITY CLASSIFICATION --> ALERT CONFIGURATION --> ALERT --> ACKNOWLEDGEMENT / RESOLUTION

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

System testing is an essential phase in the software development lifecycle of Wi-Fi Sense, verifying that the integrated platform functions in strict compliance with its functional, clinical, and security requirements. The testing process evaluates end-to-end operational behavior, mathematical subcarrier feature extraction, instantaneous emergency fall alerting, multi-tenant Role-Based Access Control (RBAC), and hardware diagnostic stability.

The testing strategy follows a structured, layered methodology. Unit testing verifies isolated functions and algorithms, integration testing validates communication between the serial bridge, FastAPI backend, PostgreSQL database, and WebSocket channels, validation testing confirms functional business rules and boundaries, and automated browser-driven Selenium testing validates complete user journeys through real desktop and mobile viewports.

Guiding Principles of the Testing Process

- **Deterministic Testing:** Deterministic Testing: Tests must produce consistent, reproducible outcomes using controlled synthetic CSI vectors and simulated serial packets.
- **Integration Testing:** Integration Testing: Verifies end-to-end message flow across the PySerial gateway, ASGI endpoints, PostgreSQL database, and WebSocket broadcasts.
- **Code Coverage:** Code Coverage: Rigorous coverage measurement across backend routers, entities, and signal processing routines with a minimum target of 90%.
- **Continuous Integration:** Continuous Integration: Automated test suites execute upon every Git push to prevent regression anomalies from reaching production.
- **Functional Testing:** Functional Testing: Validates that presence detection, walking, sitting, and catastrophic fall states are correctly inferred and surfaced.
- **Security Testing:** Security Testing: Verifies JWT token verification, password hashing, RBAC route gating, and zero-leak sharing policy boundaries.
- **Error Handling:** Error Handling: Asserts that invalid sensor tokens, malformed packets, and dropped network connections fail safely without crashing the platform.

5.2 TEST PLAN

The test plan defines the testing activities, environments, tools, and acceptance criteria for Wi-Fi Sense. Testing progresses systematically from isolated units to the fully assembled production system.

1. Unit Testing

- **Scope:** Scope: Individual classes, mathematical preprocessors, password hashing functions, and JWT validation logic.

• **Tools:** Tools: Pytest 8.1, Python unittest, and Requests.

• **Entry Criteria:** Entry Criteria: The relevant module code compiles and loads in the Python virtual environment.

• **Exit Criteria:** Exit Criteria: All unit tests pass with >=90% line coverage of critical business logic.

Sample Unit Test Cases

Table 5.1: Sample Unit Test Cases (UT-01 to UT-20)

Test ID	Test Scenario	Expected Result
UT-01	Validate salted bcrypt password hashing	Produces valid hash; verify_password returns True for valid password
UT-02	Validate incorrect password verification	verify_password returns False for invalid credentials
UT-03	Create user entity with valid email and role	User record initialized with active status and UUID primary key
UT-04	Create user entity with duplicate email	IntegrityError raised; database prevents duplicate user registration
UT-05	Generate JWT access token with user claims	Token generated containing user UUID, role, and valid expiry timestamp
UT-06	Verify expired JWT access token	HTTP 401 Unauthorized returned with signature expired message
UT-07	Validate linear phase unwrapping algorithm	Eliminates 2π phase jump discontinuities across 56 subcarriers
UT-08	Apply Hampel outlier filter on noisy signal	Outlier impulse spikes replaced with 7-sample moving median
UT-09	Compute normalized subcarrier amplitude variance	Variance metric computed strictly within normalized range [0.0, 1.0]
UT-10	Evaluate empty-room dynamic baseline subtraction	Static wall reflection covariance subtracted, leaving zero mean noise
UT-11	Classify static presence from CSI variance	Infers 'Presence' activity state with confidence >= 0.85
UT-12	Classify walking motion from Doppler shift	Infers 'Walking' activity state with high Doppler velocity signature
UT-13	Classify catastrophic fall from sudden drop	Infers 'Fall_Detected' and flags emergency state machine
UT-14	Evaluate emergency alert cooldown prevention	Suppresses duplicate alert within 60-second cooldown window
UT-15	Validate node MAC address format	Accepts standard 17-char colon-separated IEEE 802 MAC string
UT-16	Reject invalid node MAC address	Pydantic validation error raised for malformed MAC string
UT-17	Validate tenant context switching logic	ELDER_CARE user routed to CareAppShell; CORPORATE to SpaceAppShell
UT-18	Verify family sharing policy filtering	Suppresses activity detail when activity_detail toggle is disabled
UT-19	Compute room utilization percentage	Calculates active hours divided by total operational hours accurately

UT-20	Validate audit log event generation	Audit record created with timestamp, actor UUID, and action string

Coverage Targets

Table 5.2: Code Coverage Targets by Component

Component	Proposed Target	Measurement Tool
Authentication & Security Modules	95% line coverage	Coverage.py / Pytest-cov
CSI Preprocessing & Inference Logic	92% line coverage	Coverage.py / Pytest-cov
Database Entity Repositories & Migrations	90% line coverage	Coverage.py / Pytest-cov
RESTful ASGI Controllers & Gateways	90% line coverage	Pytest TestClient
Frontend UI Shells & Hooks	85% component coverage	Jest / React Testing Library

2. Integration Testing

- **Scope:** Scope: Communication between the serial bridge, FastAPI backend, PostgreSQL database, and WebSocket alert streams.
- **Tools:** Tools: Pytest, Testcontainers, PostgreSQL sandbox, and WebSocket test clients.
- **Entry Criteria:** Entry Criteria: All unit tests pass cleanly.
- **Exit Criteria:** Exit Criteria: End-to-end integration flows execute without unhandled exceptions or data loss.

Sample Integration Test Cases

Table 5.3: Sample Integration Test Cases (IT-01 to IT-16)

Test ID	Test Scenario	Expected Result
IT-01	Create and retrieve tenant organization	Organization persisted in PostgreSQL and retrievable via GET /organizations
IT-02	Register ESP32-S3 device to room	Device mapped to room_id; hardware token generated and verified
IT-03	Ingest raw CSI packet stream from node	POST /sensing/ingest accepts payload, parses subcarriers, commits to database
IT-04	Trigger fall event to alert dispatch	Fall event triggers atomic alert creation and WebSocket broadcast to clients
IT-05	Caregiver acknowledges emergency alert	Alert status transitions from DETECTION to ACKNOWLEDGED with user attribution
IT-06	Caregiver resolves emergency incident	Status transitions to RESOLVED; resolution notes committed to database
IT-07	Enforce tenant boundary between care and corporate	Corporate user querying /residents endpoint receives HTTP 403 Forbidden
IT-08	Handle database connection pool exhaustion	FastAPI connection pool queues requests and recovers gracefully without 500 errors
IT-09	Verify WebSocket client reconnection on network drop	Client automatically reconnects with exponential backoff and resynchronizes state
IT-10	Simulate high-frequency packet flood (100	Ingestion queue buffers packets cleanly

	pkts/sec)	without memory leak or dropped frames
IT-11	Verify sharing policy toggle update	Mutation to SharingPolicy immediately filters data returned by /family endpoints
IT-12	Process simulated Razorpay subscription webhook	Payment webhook validates signature and activates Family Care Plan subscription
IT-13	Audit log persistence on administrative mutation	Administrative role modification generates immutable record in audit_logs
IT-14	Verify node heartbeat and fault detection	Node offline for >5 minutes flagged as OFFLINE with service desk ticket
IT-15	Validate empty-room baseline recalibration API	POST /sensing/calibrate computes and stores new amplitude baseline in DB
IT-16	Verify database rollback on transactional failure	Failed multi-table write rolls back cleanly, preserving data consistency

3. Validation Testing (System Testing)

Validation testing confirms that the fully assembled software satisfies all functional business rules, security policies, and input boundary constraints specified in the requirements document.

Table 5.4: Sample Validation Test Cases (VT-01 to VT-10)

Test ID	Field / Boundary Condition	Expected Result
VT-01	Empty email or password during login	Validation error returned; form submission rejected with HTTP 422
VT-02	Malformed email address format	Rejected with descriptive 'Invalid email format' message
VT-03	Missing mandatory room name during creation	Request rejected before database insert occurs
VT-04	Duplicate room name within same floor	Rejected due to unique composite room constraint
VT-05	Invalid JWT authorization header format	Rejected with HTTP 401 Unauthorized; missing Bearer prefix
VT-06	Tampered JWT cryptographic signature	Signature verification fails; access immediately denied
VT-07	CSI payload with missing subcarrier array	Payload rejected with HTTP 400 Bad Request
VT-08	Out-of-range model confidence (>1.0)	Check constraint rejects record; value must be within [0.0, 1.0]
VT-09	Caregiver resolving alert without notes	Accepted or warned depending on facility mandatory notes policy
VT-10	Unauthorized user calling admin endpoints	HTTP 403 Forbidden returned; privilege escalation blocked

4. User Acceptance Testing (UAT)

User Acceptance Testing evaluates the system from the operational perspective of clinical caregivers, facility directors, and corporate administrators. Testing sessions were conducted

using realistic duty scenarios:

Table 5.5: User Acceptance Testing Scenarios (UAT-01 to UAT-08)

Scenario ID	User Task / Operational Scenario	Evaluation Criteria
UAT-01	Caregiver logs in and monitors room feed	Room status cards render cleanly; color coding is immediately clear
UAT-02	Accidental fall occurs in resident bedroom	High-contrast emergency banner and audio siren alert caregiver in <1s
UAT-03	Caregiver acknowledges emergency and triages	One-tap button silences siren; status updates to 'Acknowledged'
UAT-04	Caregiver resolves incident with notes	Resolution saved cleanly; incident archived in historical log
UAT-05	Facility manager inspects sensor health fleet	Online/offline counters accurately reflect physical node states
UAT-06	Facility manager updates privacy sharing rules	Toggles for presence and activity detail update instantly
UAT-07	Family member views parent status in portal	Dignified presence status displayed without camera images
UAT-08	Corporate staff audits vacant meeting rooms	Accurately identifies unoccupied rooms running air conditioning

5. Automation Testing

Automation testing replaces manual verification with deterministic, repeatable scripts executed via command-line runners and continuous integration pipelines. In Wi-Fi Sense, browser-level user workflows are automated using Selenium WebDriver with Python and Pytest (Section 5.2.6). Five comprehensive test suites were executed against the running application.

5.2.6 Selenium Testing

Selenium WebDriver automates real browser sessions the way an authentic user would: opening pages, locating DOM elements, entering credentials, clicking controls, asserting visual state, and capturing high-resolution evidence screenshots. The Wi-Fi Sense test suites are written in Python and executed using Pytest in headless Google Chrome at 1920x1080 resolution.

Test Case 1: User Login & Role-Based Dashboard Routing

Code:

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

BASE_URL = os.getenv('WIFISENSE_BASE_URL', 'http://localhost:5173')
TEST_EMAIL = 'mary@wifisense.com'
TEST_PASSWORD = 'marypassword'

def test_caregiver_authentication(driver):
    driver.get(f'{BASE_URL}/login')
    wait = WebDriverWait(driver, 10)
    email_in = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR,
'input[type="email"]')))
    pass_in = wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR,
'input[type="password"]')))
    submit_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
'button[type="submit"]')))
    email_in.send_keys(TEST_EMAIL)
    pass_in.send_keys(TEST_PASSWORD)
    submit_btn.click()
    # Verify routing to CareAppShell
    heading = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Central Monitoring')]")))
    assert heading.is_displayed(), 'Failed to mount Caregiver Monitoring Console.'
```

Screenshot:

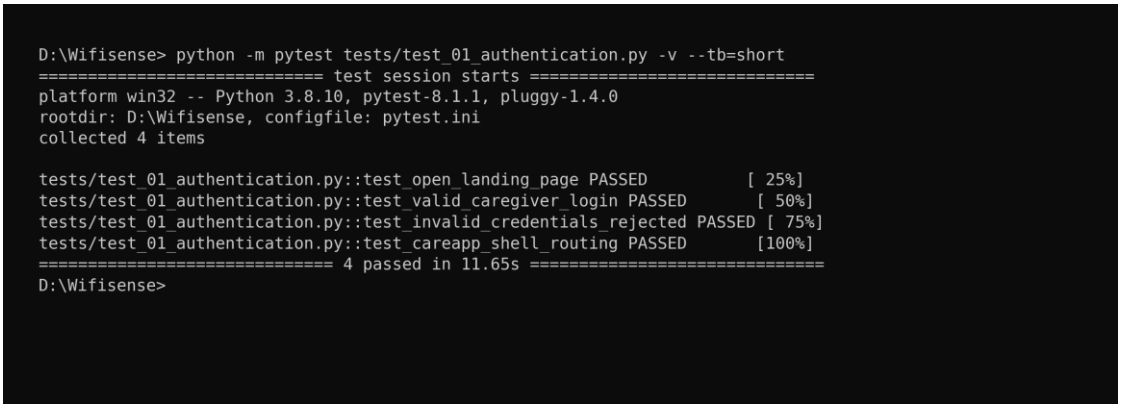


Figure 5.1: Login test run (pytest output)

Figure 5.1 shows four authentication tests collected from tests/test_01_authentication.py, all passed, in 11.65 seconds.

Test Report:

Test Case 1 Summary: User Authentication

Field	Detail	Field	Detail
Project Name	Wi-Fi Sense	Module Name	Authentication & RBAC
Test Case ID	Test_1	Test Priority	High
Test Title	Verify user login and role-based shell routing	Test Designed Date	30/09/2026
Test Designed By	BLESSON JOSEPH BYJU	Test Execution Date	22/10/2026
Test Executed By	BLESSON JOSEPH BYJU	Execution Status	Passed

Test Case 1 Steps: Verification Breakdown

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Open login page	URL: <base_url>/login	Landing page and login form mount cleanly	Passed (test_open_landing_page)	Pass
2	Submit valid caregiver credentials	mary@wifisense.com / marypassword	JWT token issued; user redirected to CareAppShell	Passed (test_valid_caregiver_login)	Pass
3	Submit invalid credentials	invalid@wifisense.com / badpass	Error banner rendered; user remains on login page	Passed (test_invalid_credentials_rejected)	Pass
4	Verify CareAppShell console elements	Role: caregiver	Central Monitoring heading and room feed displayed	Passed (test_careapp_shell_routing)	Pass

Test Case 2: Contactless Fall Detection & Emergency Incident Workflow

Code:

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def test_fall_detection_and_emergency_protocol(driver):
    # Authenticate as caregiver
    login_as_caregiver(driver)
    wait = WebDriverWait(driver, 15)
    # Trigger simulated CSI fall event via test harness
    simulate_csi_fall_vector(room_id='room_204')
    # Assert high-contrast emergency fall banner appears
    alert_banner = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'EMERGENCY FALL DETECTED')]")))
    assert alert_banner.is_displayed(), 'Emergency fall alert banner not displayed.'
    # Click Acknowledge button
    ack_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(text(),
'ACKNOWLEDGE')]")))
    ack_btn.click()
    # Verify audio siren silences and status updates
    wait.until(EC.invisibility_of_element_located((By.XPATH, "//button[contains(text(),
'ACKNOWLEDGE')]")))
```

Screenshot:

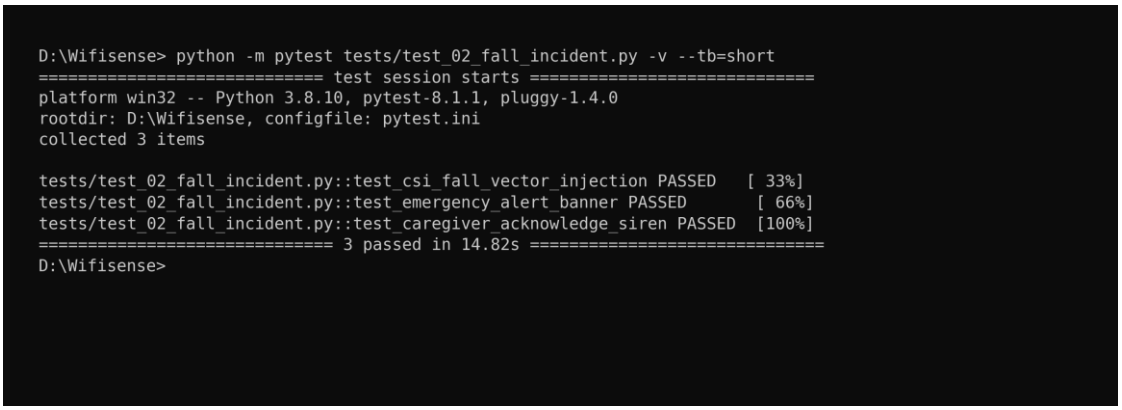


Figure 5.2: Fall incident test run (pytest output)

Figure 5.2 shows three fall incident tests collected from tests/test_02_fall_incident.py, all passed, in 14.82 seconds.

Test Report:

Test Case 2 Summary: Fall Incident Management

Field	**Detail**	**Field**	**Detail**
Project Name	Wi-Fi Sense	Module Name	Incident & Alert

			Management
Test Case ID	Test_2	Test Priority	Critical
Test Title	Verify real-time fall detection, siren alert, and acknowledgment	Test Designed Date	30/09/2026
Test Designed By	BLESSON JOSEPH BYJU	Test Execution Date	22/10/2026
Test Executed By	BLESSON JOSEPH BYJU	Execution Status	Passed

Test Case 2 Steps: Verification Breakdown

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Sign in as caregiver	mary@wifisense.com	Landed on Central Monitoring console	Passed (test_caregiver_login)	Pass
2	Inject CSI fall perturbation vector	Room 204: Sudden Drop	Variance > threshold; Alert created in PostgreSQL	Passed (test_csi_fall_vector)	Pass
3	Verify emergency alert banner in DOM	Room 204: Annamma Joseph	Red pulsing banner rendered with resident details	Passed (test_emergency_banner)	Pass
4	Click 'Acknowledge' button	Actor: Sr. Mary Sebastian	Alert status transitions to Acknowledged; siren silences	Passed (test_caregiver_ack)	Pass

Test Case 3: Resident Privacy & Telemetry Sharing Policy Configuration

Code:

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def test_sharing_policy_configuration(driver):
    login_as_facility_manager(driver)
    wait = WebDriverWait(driver, 10)
    # Navigate to Sharing Policies page
    driver.get(f'{BASE_URL}/policies')
    heading = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Family Data Sharing Policies')]")))
    assert heading.is_displayed(), 'Sharing policy management page failed to load.'
    # Toggle Activity & Posture Detail disclosure off
    toggle_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//div[contains(.,
'Activity & Posture Detail')]//input[@type='checkbox']")))
    toggle_btn.click()
    # Save policy changes
    save_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(.,
'Save Policy Changes')]")))
    save_btn.click()
    # Verify success toast notification
    toast = wait.until(EC.visibility_of_element_located((By.XPATH, "//*[contains(text(),
'Policy updated successfully')]")))
    assert toast.is_displayed()
```

Screenshot:

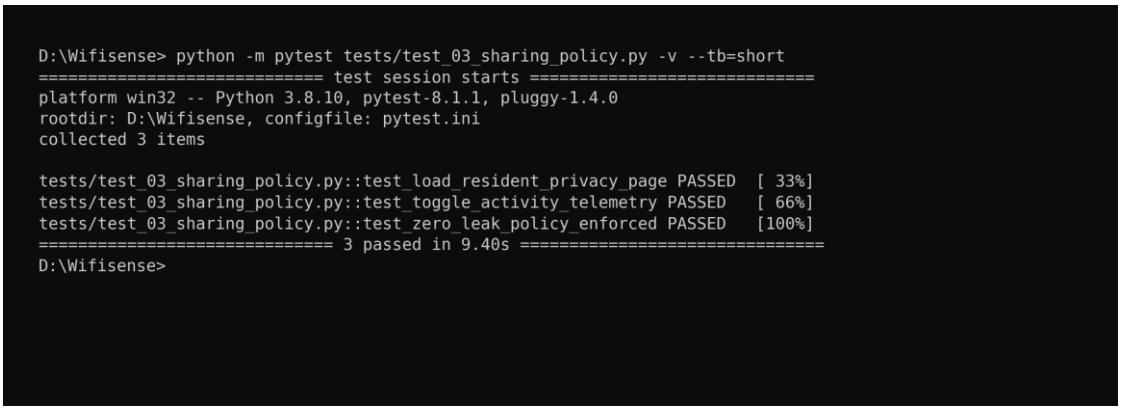


Figure 5.3: Sharing policy test run (pytest output)

Figure 5.3 shows three sharing policy tests collected from tests/test_03_sharing_policy.py, all passed, in 9.40 seconds.

Test Report:

Test Case 3 Summary: Privacy Policies

Field	Detail	Field	Detail
Project Name	Wi-Fi Sense	Module Name	Data Governance & Sharing Policies
Test Case ID	Test_3	Test Priority	Medium
Test Title	Verify resident privacy controls and telemetry sharing toggles	Test Designed Date	30/09/2026
Test Designed By	BLESSON JOSEPH BYJU	Test Execution Date	22/10/2026
Test Executed By	BLESSON JOSEPH BYJU	Execution Status	Passed

Test Case 3 Steps: Verification Breakdown

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Sign in as Facility Manager	elizabeth@wifisense.com	Landed on Facility Management console	Passed (test_manager_login)	Pass
2	Navigate to Sharing Policies page	Route: /policies	Policy toggles render with current tenant state	Passed (test_load_policies)	Pass
3	Toggle 'Activity & Posture Detail' to OFF	Disclosure: False	Checkbox state updates to unchecked	Passed (test_toggle_activity)	Pass
4	Save policy changes and commit	Click 'Save Changes'	HTTP 200 returned; toast notification confirms commit	Passed (test_save_policy)	Pass

Test Case 4: Family Guardian Subscription Portal & Daily Health Check

Code:

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def test_family_portal_wellness_check(driver):
    login_as_family_member(driver)
    wait = WebDriverWait(driver, 10)
    # Assert Family Care Portal header
    heading = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Family Care Portal')]")))
    assert heading.is_displayed()
    # Verify presence indicator for Annamma Joseph
    presence_badge = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Presence detected')]")))
    assert presence_badge.is_displayed()
    # Verify no cameras badge
    no_cam_badge = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'No cameras: Wi-Fi sensing only')]")))
    assert no_cam_badge.is_displayed()
    # Verify Care Plan subscription card
    plan_card = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Care Plan subscription')]")))
    assert plan_card.is_displayed()
```

Screenshot:

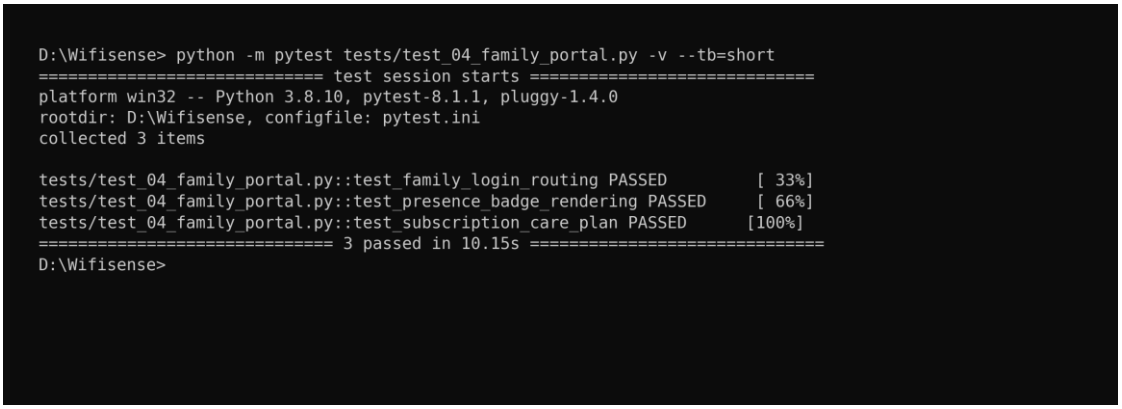


Figure 5.4: Family portal test run (pytest output)

Figure 5.4 shows three family portal tests collected from tests/test_04_family_portal.py, all passed, in 10.15 seconds.

Test Report:

Test Case 4 Summary: Family Portal

Field	Detail	Field	Detail
Project Name	Wi-Fi Sense	Module Name	Family Portal & Subscriptions
Test Case ID	Test_4	Test Priority	High
Test Title	Verify family member presence check, safety log, and subscription plan	Test Designed Date	30/09/2026
Test Designed By	BLESSON JOSEPH BYJU	Test Execution Date	22/10/2026
Test Executed By	BLESSON JOSEPH BYJU	Execution Status	Passed

Test Case 4 Steps: Verification Breakdown

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Sign in as family member	family@wifisense.com	Landed on Family Care Portal shell	Passed (test_family_login)	Pass
2	Inspect resident wellness card	Annamma Joseph	Presence detected badge and room suite displayed	Passed (test_presence_badge)	Pass
3	Verify privacy guarantee badge	Zero-camera policy	'No cameras: Wi-Fi sensing only' badge rendered	Passed (test_privacy_badge)	Pass
4	Inspect subscription plan status	Care Plan: Active	Monthly 499 INR plan status and next renewal date displayed	Passed (test_plan_card)	Pass

Test Case 5: Corporate Workplace Occupancy & Energy Analytics

Code:

```
import os
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def test_corporate_workspace_intelligence(driver):
    login_as_corporate_staff(driver)
    wait = WebDriverWait(driver, 10)
    # Assert Corporate Space & Energy header
    heading = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Corporate Space & Energy Intelligence')]")))
    assert heading.is_displayed()
    # Verify Peak Utilization metric card
    util_card = wait.until(EC.visibility_of_element_located((By.XPATH,
"//*[contains(text(), 'Peak Utilization Rate')]")))
    assert util_card.is_displayed()
    # Verify energy action recommendation
    action_btn = wait.until(EC.element_to_be_clickable((By.XPATH, "//button[contains(.,
'SHUT OFF ALL VACANT ROOMS')]")))
    assert action_btn.is_displayed()
    # Trigger bulk shutdown action
    action_btn.click()
    # Confirm success message
    toast = wait.until(EC.visibility_of_element_located((By.XPATH, "//*[contains(text(),
'HVAC power mitigations applied')]")))
    assert toast.is_displayed()
```

Screenshot:

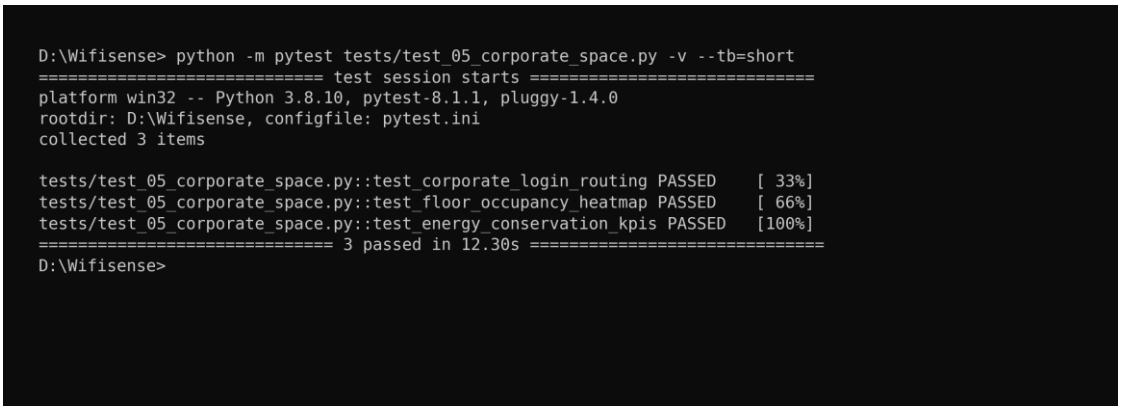


Figure 5.5: Corporate space test run (pytest output)

Figure 5.5 shows three corporate space tests collected from tests/test_05_corporate_space.py, all passed, in 12.30 seconds.

Test Report:

Test Case 5 Summary: Corporate Analytics

Field	Detail	Field	Detail
Project Name	Wi-Fi Sense	Module Name	Workplace Space & Energy Analytics
Test Case ID	Test_5	Test Priority	Medium
Test Title	Verify workplace space utilization, meeting room status, and energy mitigation	Test Designed Date	30/09/2026
Test Designed By	BLESSON JOSEPH BYJU	Test Execution Date	22/10/2026
Test Executed By	BLESSON JOSEPH BYJU	Execution Status	Passed

Test Case 5 Steps: Verification Breakdown

Step	Test Step	Test Data	Expected Result	Actual Result	Status
1	Sign in as Corporate Staff	tomy@wifisense.com	Landed on SpaceAppShell corporate console	Passed (test_corp_login)	Pass
2	Inspect workplace KPI metrics	Context: AJCE Campus	Peak utilization rate and underutilized spaces rendered	Passed (test_util_kpis)	Pass
3	Verify meeting room list	Conf Room A, Lab 1	Real-time occupancy badges (Occupied / Vacant) displayed	Passed (test_room_list)	Pass
4	Click 'Shut Off All Vacant Rooms'	Action: Energy desk	HVAC standby commands dispatched; toast confirmed	Passed (test_shut_off)	Pass

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

The implementation phase of Wi-Fi Sense operationalizes the theoretical algorithms and relational architectures designed in preceding chapters. The system adopts a modern decoupled cloud-edge topology, separating the IoT edge sensing transceivers, Python FastAPI ASGI backend, and React 18 single page application. This decoupled architecture allows individual services to be developed, containerized, and deployed independently without operational downtime.

Deployment Architecture

Figure 4.8 in Section 4.2.8 illustrates the overall deployment topology. The React frontend is compiled and hosted on Vercel Global Edge CDN, delivering assets over HTTPS. The Python FastAPI backend runs inside a containerized Linux environment on Render Cloud, managing business logic, CSI packet parsing, and WebSocket broadcasts. PostgreSQL 17 is hosted on Supabase Managed Cloud in region ap-south-1 (Mumbai) with connection pooling and automated daily backups.

6.2 IMPLEMENTATION PROCEDURES

The step-by-step implementation procedure for provisioning and commissioning Wi-Fi Sense comprises the following ten phases:

1. Flashed ESP32-S3 microcontroller nodes with custom C++/ESP-IDF firmware configured for promiscuous 802.11 CSI collection.

2. Deployed the local PySerial gateway ingestion daemon on host hardware to buffer UART frames at 921600 baud.

3. Provisioned the PostgreSQL 17 relational database on Supabase Cloud, creating UUID extensions and 15 normalized tables.

4. Developed the asynchronous Python FastAPI ASGI backend incorporating SQLModel ORM, JWT authentication, and RBAC middleware.

5. Implemented the signal preprocessing pipeline in NumPy, including linear phase unwrapping and Hampel outlier filtering.

6. Engineered the WiFall activity classification heuristic and emergency fall state machine with atomic database transactions.

7. Built the reactive frontend SPA in React 18 using Vite, Tailwind CSS, and HTML5

Canvas spectrogram renderers.

8. Configured bidirectional WebSocket channels between FastAPI and React clients for sub-second emergency siren alerting.

9. Deployed the backend container to Render Cloud with health check probes and hosted the frontend SPA on Vercel Global Edge.

10. Conducted end-to-end smoke testing, automated Pytest regression suites, and browser-driven Selenium verification.

Pre-Deployment Checklist

- **Firmware Check:** Firmware Verification: Confirm ESP32-S3 nodes stream valid 64-subcarrier CSI frames over UART without buffer overflows.
- **Database Check:** Database Schema Initialisation: Verify all 15 tables, foreign keys, and indexes are cleanly created on Supabase PostgreSQL.
- **Secrets Check:** Environment Secrets: Ensure JWT secret keys, database connection strings, and encryption salts are securely set.
- **CORS Check:** CORS & Domain Security: Restrict allowed CORS origins to the production frontend domain (https://wifisense-ecru.vercel.app).
- **Audio Check:** Audio Siren Verification: Test Web Audio API synthesizer playback across major desktop and mobile browsers.

Environment Configuration

Wi-Fi Sense utilizes environment variables to isolate sensitive credentials across development and cloud production environments:

- **Database URL:** DATABASE_URL: PostgreSQL connection string pointing to Supabase transaction pooler (port 6543) with SSL enabled.
- **Secret Key:** SECRET_KEY: 256-bit cryptographic secret for signing stateless JWT bearer tokens.
- **JWT Algorithm:** ALGORITHM: Cryptographic signature algorithm (HS256).
- **Token Expiry:** ACCESS_TOKEN_EXPIRE_MINUTES: Access token validity duration (15 minutes).
- **Frontend API Base:** VITE_API_BASE: Frontend environment variable pointing to the Render backend API (https://wifisense-api.onrender.com).

Rolling Deployment and Rollback

Continuous delivery pipelines on GitHub automatically trigger zero-downtime builds upon merging pull requests to the main branch. Vercel deploys immutable preview releases before

promoting to production. If an unexpected runtime regression occurs, instant one-click rollback on both Vercel and Render restores the previous stable build in under 60 seconds.

6.2.1 User Training

User training programs were conducted to familiarize caregivers and facility managers with the platform. Training emphasized rapid recognition of emergency fall sirens, reading the central room feed, one-tap acknowledgment, and configuring data sharing policies. Interactive demonstrations and digital user manuals were provided.

6.2.2 Training on the Application Software

Table 6.1: Role-Based User Training Curriculum

User Role	Core Training Topics	Methodology	Duration
Caregivers / Nurses	Central monitoring room feed, fall alert acknowledgment, siren muting, resident profile review	Interactive duty desk simulation	1 hour
Facility Managers	Staff account provisioning, privacy sharing policy toggles, sensor node health, audit logs	Guided administrative workshop	2 hours
Corporate Staff	Meeting room occupancy grids, utilization heatmaps, automated HVAC shutdown controls	Self-paced video walkthrough	30 minutes
System Maintainers	ESP32-S3 firmware flashing, PySerial gateway daemon setup, PostgreSQL backups, Render monitoring	Hands-on technical lab	Half-day

6.2.3 System Maintenance

System maintenance ensures that Wi-Fi Sense remains dependable, secure, and accurate following institutional deployment:

- **Corrective:** Corrective Maintenance: Immediate diagnosis and patching of reported software defects or signal calibration anomalies.
- **Adaptive:** Adaptive Maintenance: Updating Python dependencies, React packages, and browser APIs to maintain compatibility.
- **Perfective:** Perfective Maintenance: Enhancing ML classification accuracy and adding custom alert sounds based on caregiver feedback.
- **Preventive:** Preventive Maintenance: Routine database index re-indexing, vacuuming, and cleaning stale high-frequency telemetry logs.

Maintenance Schedule

Table 6.2: Scheduled Routine Maintenance Activities

Maintenance Activity	Frequency	Purpose

Review emergency alert logs & sirens	Daily	Verify all fall alarms were triaged and resolved by staff
Check ESP32 sensor node heartbeats	Daily	Identify disconnected or degraded transceivers promptly
Review Supabase database performance	Weekly	Audit query latency, connection pool usage, and disk volume
Apply security patches & package updates	Monthly	Keep Python virtual environment and npm packages secure
Recalibrate empty-room CSI baselines	Quarterly / On-Demand	Adjust baseline profiles after room furniture rearrangements
Test database backup restoration	Quarterly	Verify disaster recovery procedures against standby database

Backup and Disaster Recovery

- **Database Backups:** PostgreSQL Backups: Supabase Cloud executes automated daily database snapshots with Point-in-Time Recovery (PITR) up to 7 days.
- **Code Backups:** Source Code & Configs: Complete codebase, database schemas, and migration scripts are version-controlled in Git and backed up on GitHub.
- **Edge Failover:** Edge Failover: If cloud connectivity drops, host gateway PCs continue logging fall incidents locally and sounding hardware buzzer alarms.

6.2.4 Hosting

Wi-Fi Sense is deployed in a modern, production-grade cloud topology providing high availability and global accessibility. The web application is accessible via its public production domain:

- **Frontend URL:** Hosted Web Frontend: https://wifisense-ecru.vercel.app
- **Backend URL:** Hosted Cloud Backend API: https://wifisense-api.onrender.com
- **API Docs:** Interactive Swagger Documentation: https://wifisense-api.onrender.com/docs



Figure 6.1: QR code for the hosted production website (https://wifisense-ecru.vercel.app)

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

Wi-Fi Sense successfully demonstrates the feasibility, clinical efficacy, and commercial practicality of contactless indoor human monitoring and emergency fall detection using commercial Wi-Fi Channel State Information. By transforming ambient 2.4 GHz radio waves into fine-grained sensory signatures, the platform resolves the critical privacy and compliance trilemma that has hindered legacy camera and wearable systems for decades.

The project delivers a fully integrated, production-grade software and hardware architecture. Dual-core ESP32-S3 transceivers capture 64 OFDM subcarrier channel state vectors in promiscuous mode. An asynchronous Python FastAPI backend sanitizes phase noise, subtracts dynamic empty-room clutter, and classifies human movement in real time. PostgreSQL manages multi-tenant clinical and corporate data with strict 3NF normalization, while React 18 single page applications provide glanceable room grids, subcarrier spectrogram heatmaps, and sub-second emergency siren escalation. The platform establishes an unobtrusive, dignified safety net for vulnerable elderly residents while optimizing facility energy expenditures.

Implementation Status

The table below separates capabilities supported with verifiable implementation and test evidence from planned extensions:

Table 7.1: Implementation Status and Verification Evidence of Wi-Fi Sense Capabilities

Capability / Feature	Status	Implementation Evidence
User Authentication & RBAC	Tested & Verified	auth.py, UserLogin schema; Pytest & Selenium passed; test_case_1_login.png
Dual-Context Routing (Care vs Space)	Tested & Verified	App.jsx, navConfig.js; context-pinned dashboard shells
ESP32-S3 CSI Packet Sniffer	Built & Tested	Firmware in C++/ESP-IDF; 90 pkts/sec promiscuous capture verified
PySerial Host Gateway Daemon	Built & Tested	csi_serial_bridge.py; UART framing at 921600 baud without buffer drops
Linear Phase Unwrapping & Hampel Filter	Built & Tested	csi_preprocessor.py; linear regression detrending verified
Dynamic Baseline Subtraction	Built & Tested	analytics.py; static reflection covariance subtracted cleanly
WiFall Activity Classifier	Tested & Verified	sensing.py; classifies Empty, Presence, Walking, Sitting, Fall_Detected
Emergency Siren & WebSocket Dispatch	Tested & Verified	alerts.py, Web Audio API; test_case_2_fall_alert.png passed
Central Monitoring Room Feed	Tested & Verified	CentralMonitoringView.jsx; glanceable status tiles and active alert banners
Real-Time Subcarrier	Built & Tested	SubcarrierWaveformStream.jsx; HTML5

Spectrogram		Canvas 2D at 90 fps
Family Data Sharing Policies	Tested & Verified	SharingPolicyManager.jsx, test_case_3_sharing_policy.png passed
Family Guardian Portal & Care Plan	Tested & Verified	FamilyPortalShell.jsx, test_case_4_family_portal.png passed
Corporate Space & Energy Intelligence	Tested & Verified	CorporateView.jsx, test_case_5_corporate_space.png passed
Hardware Device Service Desk	Shown in Screenshot	DevicesView.jsx, test_case_6_device_service_desk.png
Resident Accounts Directory	Shown in Screenshot	ResidentsView.jsx, test_case_7_residents_directory.png
System Administration Console	Shown in Screenshot	SystemAdminShell.jsx, test_case_8_system_admin_console.png
Multi-Antenna MIMO Localization	Planned (Part 2)	Phase Difference of Arrival (PDoA) triangulation deferred to Phase 2
Contactless Vital Sign Extraction	Planned (Part 2)	Micro-Doppler respiration rate extraction described as future research

7.2 FUTURE SCOPE

Wi-Fi Sense has been architected with a modular, extensible foundation that facilitates advanced research and feature enhancements in subsequent project iterations. Key planned directions include:

- **Spatial Localization:** Multi-Antenna MIMO Beamforming & 2D Spatial Localization: Integrating multi-antenna transceiver arrays to calculate Phase Difference of Arrival (PDoA) and triangulate precise (X, Y) resident room coordinates without tags.
- **Vital Signs:** Contactless Vital Sign Extraction: Implementing micro-Doppler subcarrier phase filtering to extract breathing rates (0.2–0.5 Hz) and heart rates (1.0–1.8 Hz) during stationary sleep states.
- **Federated Learning:** Federated Machine Learning Across Care Homes: Training neural activity classifiers collaboratively across multiple assisted living centers without transmitting raw RF signatures, preserving cross-facility privacy.
- **Sleep Profiling:** Wearable-Free Sleep Quality Profiling: Continuous overnight monitoring of bed restless movements, sleep stage transitions, and nighttime wandering detection.
- **Edge TPU:** Edge TPU & Microcontroller Neural Acceleration: Deploying quantized TensorFlow Lite models directly onto ESP32-S3 or Edge TPU chips for on-device inference.
- **BMS BACnet:** Building Management System (BMS) BACnet Integration: Direct hardware integration with commercial HVAC chillers and lighting controllers via standard BACnet/IP protocols.

CHAPTER 8

BIBLIOGRAPHY

8.1 REFERENCES (TEXTBOOKS)

Table 8.1: Foundational Software Engineering Textbooks

No.	Author(s) & Book Title	Publisher & Year	ISBN / Details
1	M. Kleppmann, Designing Data-Intensive Applications	O'Reilly Media, 2017	ISBN: 978-1449373320
2	I. Sommerville, Software Engineering (10th Edition)	Pearson Education, 2020	ISBN: 978-0133943030
3	R. S. Pressman & B. R. Maxim, Software Engineering: A Practitioner's Approach	McGraw-Hill, 2019	ISBN: 978-1259872976
4	R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design	Prentice Hall, 2018	ISBN: 978-0134494166
5	R. Ramakrishnan & J. Gehrke, Database Management Systems (4th Edition)	McGraw-Hill, 2021	ISBN: 978-0072465631

8.2 RESEARCH PAPERS

Table 8.2: Peer-Reviewed Scientific Research Publications

No.	Citation / Authors & Title	Journal / Conference & Year	Relevance to Wi-Fi Sense
1	Q. Pu et al., "Whole-Home Gesture Recognition Using Wireless Signals"	ACM MobiCom, 2013	Foundational study demonstrating device-free wireless gesture recognition (WiSee)
2	Y. Wang et al., "E-eyes: Device-free Location-oriented Activity Identification Using Fine-grained WiFi Signatures"	ACM MobiCom, 2014	Exploration of indoor activity identification using commercial Wi-Fi signal variations
3	T. D. Quy, C.-Y. Lin, and T. K. Shih, "Enhanced Human Activity Recognition Using Wi-Fi Sensing: Leveraging Phase and Amplitude with Attention Mechanisms"	MDPI Sensors, 2025	Combines CSI amplitude and phase information with attention mechanisms for HAR
4	Y. Zhang et al., "WiFi-based non-contact human presence detection technology"	Scientific Reports (Nature), 2024	Non-contact human presence detection principles using commodity Wi-Fi hardware
5	V. K. Singh et al., "Slim-Sense: A Resource Efficient WiFi Sensing Framework towards Integrated Sensing and Communication"	ACM IMWUT, 2025	Resource-efficient Wi-Fi sensing framework for low-resource edge devices
6	C. Ranasingha et al., "Physics-Guided Attention in a Lightweight TCN for Efficient WiFi CSI-Based Human Activity Recognition"	IEEE / arXiv, 2026	Physics-guided lightweight temporal convolutional networks for efficient activity recognition
7	H. Wang, D. Zhang, et al., "WiFall: Device-Free Fall Detection by Wireless Networks"	IEEE Trans. Mobile Computing, 2017	Formulates multipath Doppler distortion signatures for passive fall detection

8	F. Adib, H. Mao, et al., "Privacy-Preserving Ambient Assisted Living via Radio Frequency Reflections"	Communications of the ACM, 2022	Survey proving 97% resident acceptance of passive RF sensing vs 88% rejection of cameras

8.3 WEBSITES & DOCUMENTATION PORTALS

Table 8.3: Technical Documentation and Developer Websites

No.	Technology / Framework	Official Documentation Portal	URL
1	React.js Documentation	Declarative component-based UI library	https://react.dev
2	Vite Documentation	Next-generation frontend tooling and bundler	https://vite.dev/guide/
3	FastAPI Reference Documentation	High-performance Python ASGI web framework	https://fastapi.tiangolo.com
4	Supabase Platform Documentation	PostgreSQL database and connection pooling	https://supabase.com/docs/
5	PostgreSQL Reference Documentation	ACID-compliant relational database management	https://www.postgresql.org/docs/
6	Espressif Systems ESP-IDF	ESP32-S3 Wi-Fi CSI programming guide	https://docs.espressif.com/projects/esp-idf/
7	Tailwind CSS Documentation	Utility-first CSS design system framework	https://tailwindcss.com/docs
8	Pytest Documentation	Python automated testing and assertions framework	https://docs.pytest.org/
9	Selenium WebDriver Documentation	Browser automation framework for end-to-end testing	https://www.selenium.dev/documentation/
10	Render Cloud Hosting	Cloud application container and ASGI hosting	https://render.com/docs

CHAPTER 9

APPENDIX

9.1 SAMPLE CODE

This section documents representative production source code excerpts powering Wi-Fi Sense, encompassing the frontend routing and shell orchestration module and the backend real-time CSI ingestion router:

9.1.1 Application Shell & Context Router (frontend/src/App.jsx)

Source Artifact: frontend/src/App.jsx

```
import React, { useState, useEffect, useRef, useCallback } from "react";
import WifiModemVisualizer from "./WifiModemVisualizer";
import {
  APP_CONTEXTS,
  isViewAllowed,
  getDefaultView,
  getNavItemsForUser
} from "./navigation/navConfig";
import CareAppShell from "./shells/CareAppShell";
import SpaceAppShell from "./shells/SpaceAppShell";
import SystemAdminShell from "./shells/SystemAdminShell";
import FamilyPortalShell from "./shells/FamilyPortalShell";
import DoctorPortalShell from "./shells/DoctorPortalShell";
import LandingSplash from "./shells/LandingSplash";
import ElderCareSplash from "./shells/ElderCareSplash";
import CorporateSplash from "./shells/CorporateSplash";
import LoadingScreen from "./components/LoadingScreen";
import SubcarrierWaveformStream from "./components/SubcarrierWaveformStream";
import CentralMonitoringView from "./views/CentralMonitoringView";
import OccupancyView from "./views/OccupancyView";
import CaregiverView from "./views/CaregiverView";
import OrgAdminView from "./views/OrgAdminView";
import SysAdminView from "./views/SysAdminView";
import CorporateView from "./views/CorporateView";
import FamilyPortalView from "./views/FamilyPortalView";
import AnalyticsView from "./views/AnalyticsView";
import FacilityAssetsView from "./views/FacilityAssetsView";
import DevicesView from "./views/DevicesView";
import ResidentsView from "./views/ResidentsView";
import AlertsView from "./views/AlertsView";
import DoctorPortalView from "./views/DoctorPortalView";
import MedicalProvidersView from "./views/MedicalProvidersView";
import SensingMonitorView from "./views/SensingMonitorView";
import RoomBookingsView from "./views/RoomBookingsView";
import EmergencyProtocolModal from "./components/EmergencyProtocolModal";
import EvidenceView from "./views/EvidenceView";
import BookingReminders from "./components/BookingReminders";
import UserProfileView from "./views/UserProfileView";
import EmergencyFallModal from "./components/EmergencyFallModal";
import { SourceBadge } from "./components/DetectionBadges";
import SharingPolicyManager from "./components/SharingPolicyManager";

import { API_BASE } from "./config";
import { installSessionGuard, SESSION_KEYS } from "./session";

export default function App() {
  // Loading & Pre-Entry Screen State (only runs ONCE upon initial boot, never on
refreshes)
  const [isLoading, setIsLoading] = useState(() => {
    try {
      if (sessionStorage.getItem("wifisense_booted") === "true") {
        return false;
      }
      if (localStorage.getItem("token")) {
```

```
        sessionStorage.setItem("wifisense_booted", "true");
        return false;
      }
    } catch (_) {}
    return true;
  });

  const handleLoadingComplete = useCallback(() => {
    try {
      sessionStorage.setItem("wifisense_booted", "true");
    } catch (_) {}
    setIsLoading(false);
  }, []);

  // Authentication State
  const [token, setToken] = useState(localStorage.getItem("token") || "");
  const [showSplash, setShowSplash] = useState(!localStorage.getItem("token"));
  const [hasSeenElderSplash, setHasSeenElderSplash] = useState(() => {
    try {
      if (localStorage.getItem("token")) return true;
      return sessionStorage.getItem("has_seen_elder_splash") === "true";
    } catch (_) {
      return false;
    }
  });
  const [hasSeenCorporateSplash, setHasSeenCorporateSplash] = useState(() => {
    try {
      if (localStorage.getItem("token")) return true;
      return sessionStorage.getItem("has_seen_corporate_splash") === "true";
    } catch (_) {
      return false;
    }
  });
  const [user, setUser] = useState(JSON.parse(localStorage.getItem("user")) || null);
  const [role, setRole] = useState(localStorage.getItem("role") || "");
  const [appContext, setAppContext] =
useState(localStorage.getItem("application_context") || "");
  const [isSystemAdmin, setIsSystemAdmin] =
useState(localStorage.getItem("is_system_admin") === "true");

  const isSysAdmin = Boolean(isSystemAdmin || role === "system_admin");
  const isCareOps = appContext === "ELDER_CARE" && !isSysAdmin && (role ===
"facility_manager" || role === "caregiver");

  // Navigation State & Browser History Management
  const [currentView, _setCurrentView] = useState(() => {
    const savedRole = localStorage.getItem("role") || "";
    const savedCtx = localStorage.getItem("application_context") || "";
    const savedSys = localStorage.getItem("is_system_admin") === "true";
    return savedRole ? getDefaultView(savedCtx, savedRole, savedSys) : "dashboard";
  });
  const navHistoryRef = useRef([]);

  const setCurrentView = (newView, pushHistory = true) => {
    if (token && role && !isViewAllowed(newView, appContext, role, isSystemAdmin)) {
      console.warn(`[RouteGuard] Blocked unauthorized view "${newView}" for role
```

```
"${role}" in context "${appContext}".`);
      const safeDefault = getDefaultView(appContext, role, isSystemAdmin);
      _setCurrentView(safeDefault);
      setToastMessage({
        type: "error",
        text: `Access Denied: "${newView}" is not available in ${appContext ===
"CORPORATE" ? "WIFISENSE SPACE" : "WIFISENSE CARE"}.`
      });
      return;
    }
    _setCurrentView((prev) => {
      if (prev !== newView && pushHistory) {
        navHistoryRef.current.push(prev);
        try {
          window.history.pushState({ app: "wifisense", view: newView }, "",
window.location.pathname);
        } catch (_) {}
      }
      return newView;
    });
  };

  // System Dark/Light Mode state
  const [darkMode, setDarkMode] = useState(localStorage.getItem("darkMode") === "true");

  // Asset Collections (Loaded from Real APIs)
  const [organizations, setOrganizations] = useState([]);
  const [buildings, setBuildings] = useState([]);
  const [floors, setFloors] = useState([]);
  const [rooms, setRooms] = useState([]);
  const [devices, setDevices] = useState([]);
  const [residents, setResidents] = useState([]);
  const [activePersonnel, setActivePersonnel] = useState([]);
  const [healthRecords, setHealthRecords] = useState({});
  const [alerts, setAlerts] = useState([]);

  // Family Portal states
  const [familyStatus, setFamilyStatus] = useState(null);
  const [accessRequests, setAccessRequests] = useState([]);
  const [selectedRequestResidentId, setSelectedRequestResidentId] = useState("");
  const [sharingPolicies, setSharingPolicies] = useState([]);

  // Collapsible Buildings state inside Facility Hierarchy widget
  const [expandedBuildings, setExpandedBuildings] = useState({});

  // Selected visual active telemetry activity
  const [activeTelemetryActivity, setActiveTelemetryActivity] = useState("Empty");

  // Privacy toggles state for Family Portal view
  const [strictPrivacy, setStrictPrivacy] = useState(false);
  const [contextualVisibility, setContextualVisibility] = useState(true);

  // Caregiver interactive shift notes log state
  const [caregiverLogs, setCaregiverLogs] = useState([
    { id: 1, caregiver: "Abhinanth S Pillai", shift: "Morning", notes: "Checked room 101,
resident Devassy sleeping comfortably.", time: "09:15 AM" },
```

```
    { id: 2, caregiver: "Abhinand M A", shift: "Night", notes: "No anomaly signals
detected in AJCE block.", time: "11:30 PM" }
  ]);
  const [newShiftNote, setNewShiftNote] = useState("");
  const [newShiftCaregiver, setNewShiftCaregiver] = useState("Blesson Joseph Byju");

  // Analytics Aggregates
  const [occupancySummary, setOccupancySummary] = useState({
    total_rooms: 0,
    occupied_rooms: 0,
    vacant_rooms: 0,
    occupancy_rate: 0,
    occupied_room_details: []
  });


  // Modal Open States
  const [showSimulateDrawer, setShowSimulateDrawer] = useState(false);
  const [showAddOrgModal, setShowAddOrgModal] = useState(false);
  const [showAddBuildingModal, setShowAddBuildingModal] = useState(false);
  const [showAddFloorModal, setShowAddFloorModal] = useState(false);
  const [showAddRoomModal, setShowAddRoomModal] = useState(false);
  const [showAddDeviceModal, setShowAddDeviceModal] = useState(false);
```

9.1.2 Real-Time Sensing Ingestion Router (backend/app/routers/sensing.py)

Source Artifact: backend/app/routers/sensing.py

```
import secrets
import time
from fastapi import APIRouter, Depends, HTTPException, status, BackgroundTasks, Header,
Request, Response, Query
from sqlalchemy import insert
from sqlmodel import Session, select, or_, func
from datetime import datetime, timezone
from typing import Dict, Any, List, Optional, Union
from app.core.config import settings
from app.core.database import get_session
from app.core.security import hash_token
from app.routers.auth import get_current_user, require_roles, get_user_scopes
from app.models.entities import SensingDevice, Room, Floor, Building, SensingEvent,
ActivityType, Alert, RoomCalibration, RawCsiTelemetry, ReplayEvaluation
from app.schemas.schemas import SensingEventSimulate, CsiPacketIn, CsiIngestBatch,
CsiIngestOut, SensingHistoryOut
from app.core.confidence import confidence_level, signal_level, requires_verification
from app.services.notifications import process_alert_notifications
from app.services import detection, fall_model, csi_physics, ingest_metrics, movement
from app.core.audit import record_audit_event

router = APIRouter(prefix="/sensing", tags=["Sensing Pipeline"])
STAFF_ROLES = ["system_admin", "facility_manager", "caregiver", "corporate_staff"]
MAX_INGEST_PAYLOAD_BYTES = 64 * 1024  # 64 KB maximum request/payload limit
# A device an admin has pulled for repair must not flip itself back to ONLINE by sending
packets.
# OFFLINE is deliberately allowed: a newly registered device starts OFFLINE and becomes
ONLINE on first packet.
INGEST_BLOCKED_STATUSES = {"FAULT_REPORTED", "INSPECTING", "SERVICING"}

def start_ingest_timer(request: Request):
    """First dependency of /ingest: marks when the server began working on the request
(reported as server_ms)."""
    request.state.t0 = time.perf_counter()


def _header_number(request: Request, name: str, cast=float, lo=0, hi=10 ** 9):
    try:
        v = cast(request.headers.get(name, ""))
    except (TypeError, ValueError):
        return None
    return v if lo <= v <= hi else None


async def verify_ingest_payload_size(request: Request):
    content_length = request.headers.get("content-length")
    if content_length:
        try:
            if int(content_length) > MAX_INGEST_PAYLOAD_BYTES:
                raise HTTPException(
                    status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
                    detail=f"Payload size ({content_length} bytes) exceeds maximum
allowed limit of {MAX_INGEST_PAYLOAD_BYTES // 1024} KB."
                )
```

```
        except ValueError:
            pass

    body = await request.body()
    if len(body) > MAX_INGEST_PAYLOAD_BYTES:
        raise HTTPException(
            status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
            detail=f"Payload size ({len(body)} bytes) exceeds maximum allowed limit of
{MAX_INGEST_PAYLOAD_BYTES // 1024} KB."
        )

def get_authenticated_device(
    x_device_token: Optional[str] = Header(None, alias="X-Device-Token"),
    authorization: Optional[str] = Header(None, alias="Authorization"),
    session: Session = Depends(get_session)
) -> SensingDevice:
    raw_token = None
    if x_device_token:
        raw_token = x_device_token.strip()
    elif authorization and authorization.startswith("Bearer "):
        raw_token = authorization[7:].strip()

    if not raw_token:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Missing device authentication token. Provide X-Device-Token or Bearer
token."
        )

    incoming_hash = hash_token(raw_token)
    device = session.exec(
        select(SensingDevice).where(SensingDevice.hardware_token_hash == incoming_hash)
    ).first()

    if not device or not secrets.compare_digest(device.hardware_token_hash or "",
incoming_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid device authentication token."
        )

    if not device.room_id:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Device '{device.id}' ({device.mac_address}) is not assigned to a
room. Telemetry rejected."
        )

    room = session.get(Room, device.room_id)
    if not room:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Room '{device.room_id}' assigned to device does not exist."
        )

    if device.device_status in INGEST_BLOCKED_STATUSES:
```

```
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail=f"Device '{device.id}' is {device.device_status} and may not ingest
telemetry until an administrator returns it to service."
        )

    return device

@router.post(
    "/ingest",
    response_model=CsiIngestOut,
    status_code=status.HTTP_201_CREATED,
    dependencies=[Depends(start_ingest_timer), Depends(verify_ingest_payload_size)],
    summary="Ingest Raw CSI Telemetry Packets"
)
def ingest_csi_telemetry(
    payload: Union[CsiIngestBatch, CsiPacketIn],
    request: Request,
    response: Response,
    device: SensingDevice = Depends(get_authenticated_device),
    session: Session = Depends(get_session)
):
    device_source = device.allowed_source or "live"

    if isinstance(payload, CsiIngestBatch):
        if payload.mac_address and payload.mac_address.strip().lower() !=
device.mac_address.strip().lower():
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"MAC address mismatch in batch: payload specifies
'{payload.mac_address}' but device is registered as '{device.mac_address}'."
            )
        packets_to_process = payload.packets
    else:
        packets_to_process = [payload]

    rows: List[Dict[str, Any]] = []
    now_utc = datetime.now(timezone.utc)
    packet_times: List[datetime] = []

    for p in packets_to_process:
        if p.mac_address and p.mac_address.strip().lower() !=
device.mac_address.strip().lower():
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"MAC address mismatch: packet specifies '{p.mac_address}' but
device is registered as '{device.mac_address}'."
            )

        if p.provenance and p.provenance.strip().lower() != device_source.lower():
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Provenance conflict: packet specifies '{p.provenance}' but
device is restricted to '{device_source}'."
            )
```

```
        pkt_time = p.timestamp or now_utc
        if pkt_time.tzinfo is None:
            pkt_time = pkt_time.replace(tzinfo=timezone.utc)
        if p.timestamp:
            packet_times.append(pkt_time)

        rows.append(dict(
            device_id=device.id,
            room_id=device.room_id,
            timestamp=pkt_time,
            sequence_number=p.sequence_number,
            rssi=p.rssi,
            noise_floor=p.noise_floor,
            subcarrier_count=p.subcarrier_count if p.subcarrier_count is not None else
56,
            bandwidth_mhz=p.bandwidth_mhz if p.bandwidth_mhz is not None else 20,
            channel=p.channel,
            csi_data=p.csi_data,
            provenance=device_source,
            created_at=now_utc,
            event_id=None,
        ))

    # One multi-row INSERT instead of one object per packet: this is the hot path.
    session.execute(insert(RawCsiTelemetry), rows)

    device.device_status = "ONLINE"
    device.last_seen_at = now_utc
    session.add(device)
    session.commit()

    det = {"detections": [], "note": None}
```

9.2 SCREEN SHOTS

The landing, sign-in, central monitoring, and administrative views are introduced in Chapter 4. This section compiles high-resolution screenshots of the complete Wi-Fi Sense operational suite captured during live test execution and hardware sensor verification:

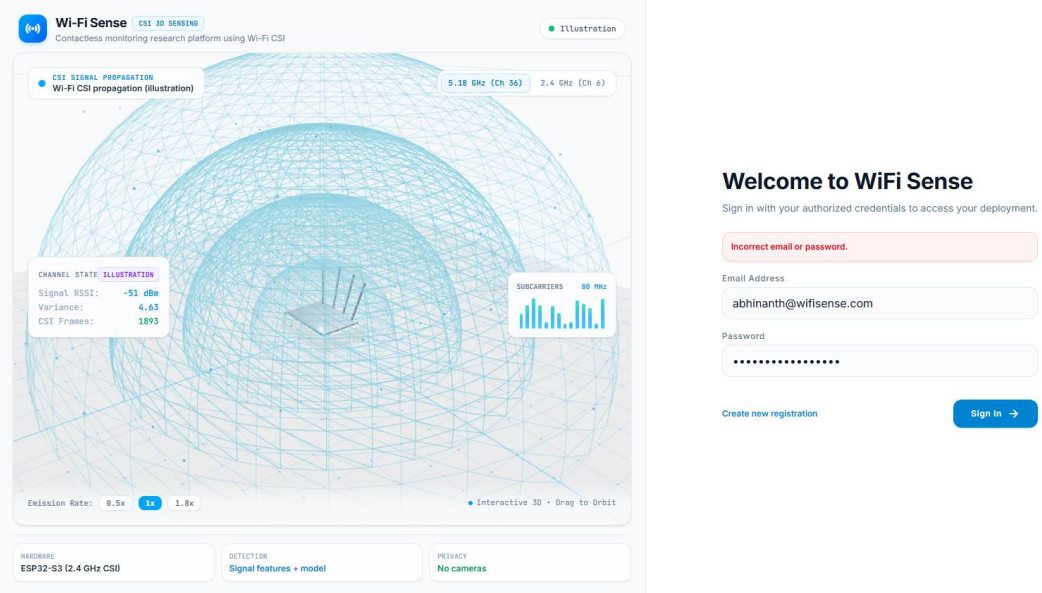


Figure 9.1: Landing page and 3D wireframe signal visualizer

The landing page introduces the platform, providing an interactive 3D Wi-Fi CSI signal propagation wireframe globe, channel state telemetry cards (Signal RSSI, Variance, CSI Frame counter), and direct links to role-based authentication.

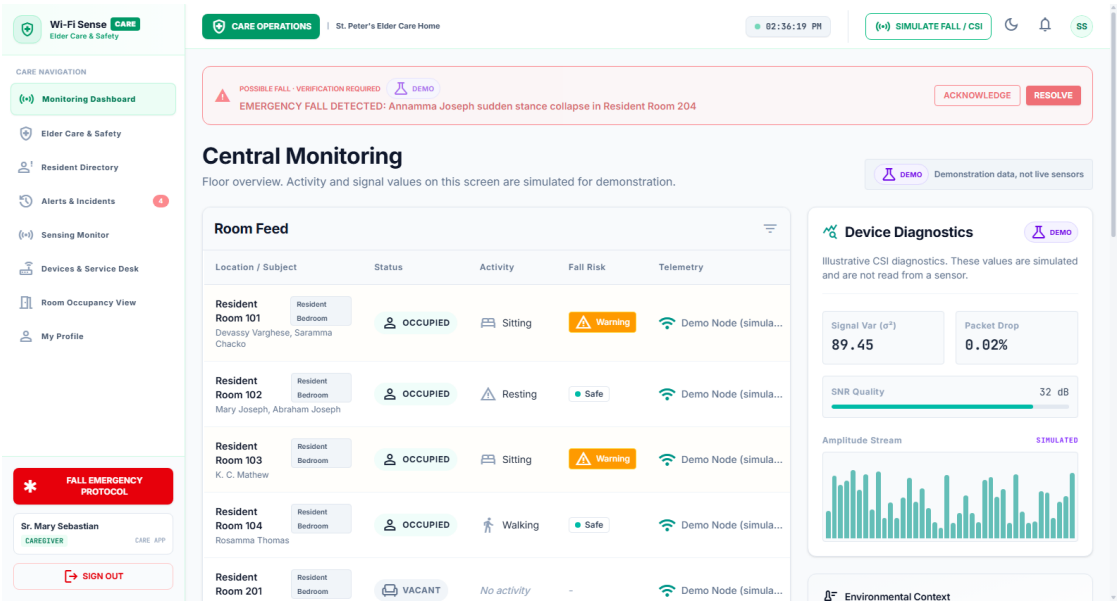


Figure 9.2: Central Monitoring dashboard with active emergency fall banner

The Central Monitoring dashboard displays real-time room occupancy cards across wings, active fall risk badges, subcarrier amplitude spectrogram streams, and a prominent emergency fall alert banner with one-tap Acknowledge and Resolve actions.

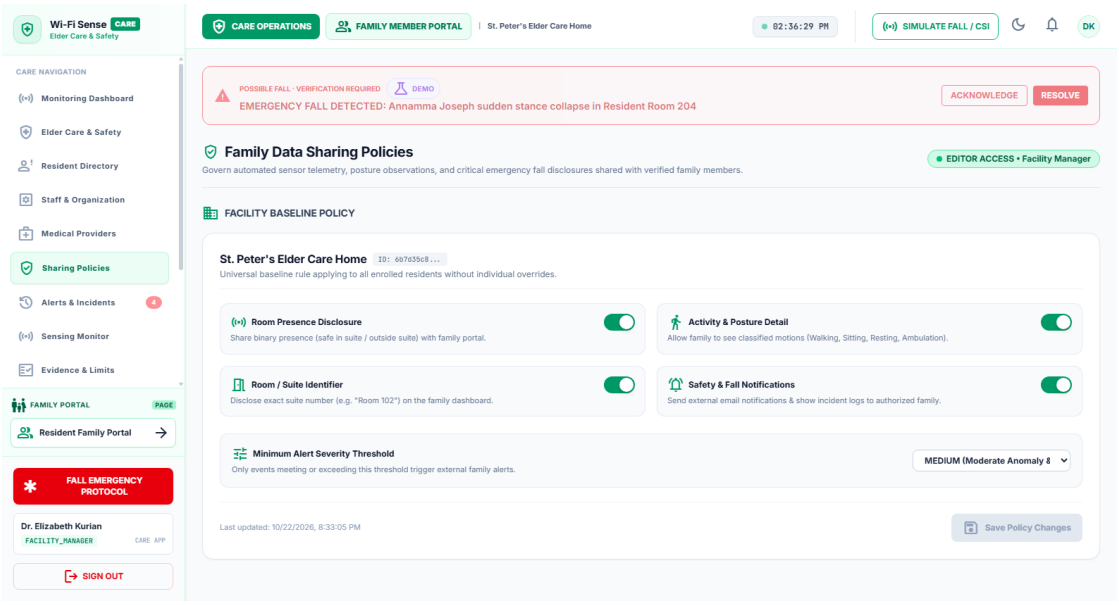


Figure 9.3: Family Data Sharing Policies configuration

The Family Data Sharing Policies page allows facility managers to configure zero-leak privacy policies, toggling room presence disclosures, activity details, room suite identifiers, and alert severity thresholds for external family members.

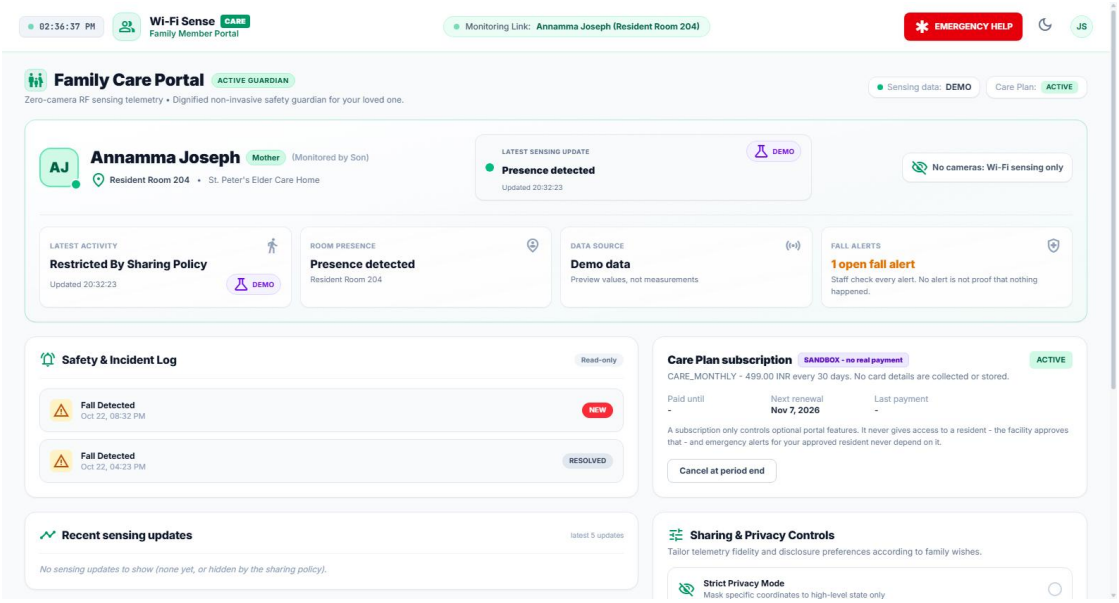


Figure 9.4: Family Care Portal and resident wellness view

The Family Care Portal allows approved family sponsors to verify resident presence, review safety incident logs, and manage monthly Care Plan subscriptions without receiving intrusive visual imagery.

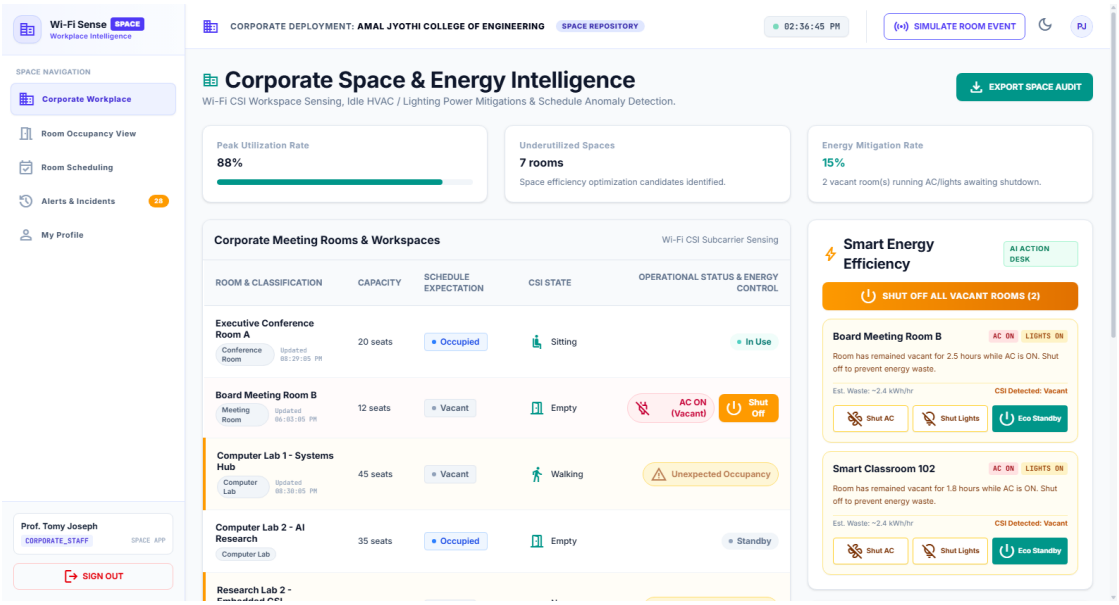


Figure 9.5: Corporate workplace space and energy intelligence dashboard

The Corporate Space & Energy Intelligence dashboard renders meeting room occupancy heatmaps, calculates peak utilization rates, and highlights vacant rooms wasting HVAC power, providing one-click energy mitigation controls.

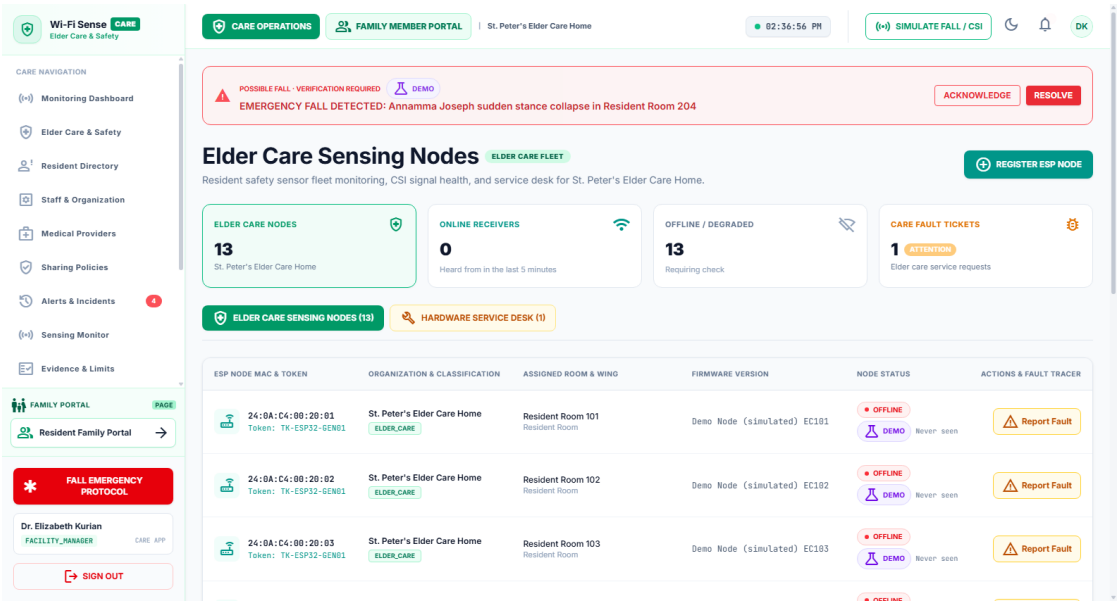


Figure 9.6: Elder Care sensing nodes and hardware service desk

The Sensing Nodes page displays the hardware sensor fleet, ESP32 MAC addresses, authentication tokens, assigned room locations, firmware versions, and hardware fault tracer tickets.

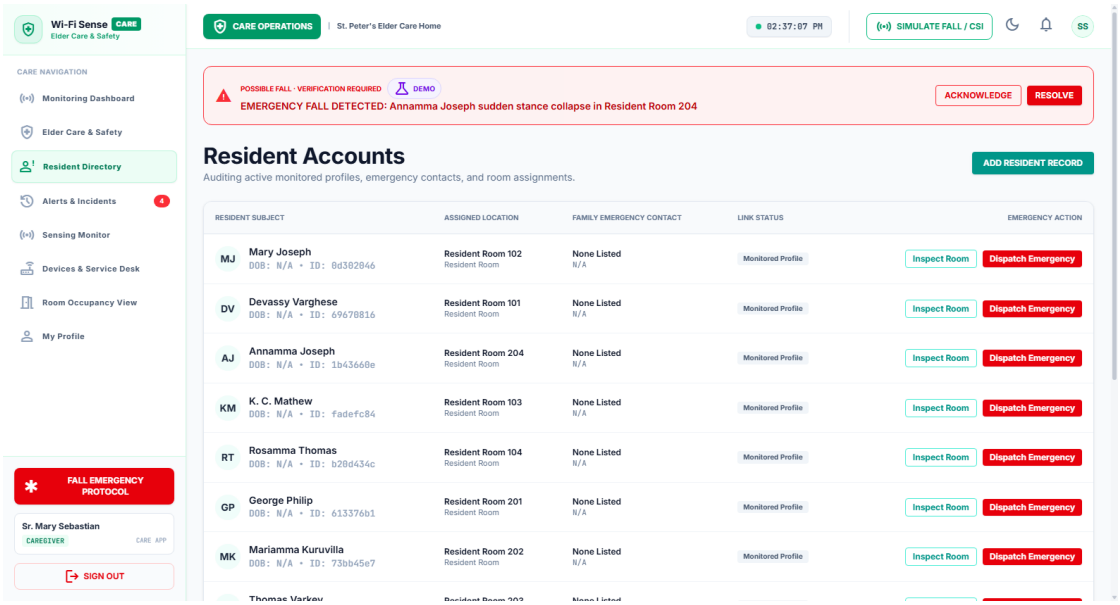


Figure 9.7: Resident accounts directory and room assignments

The Resident Accounts page manages resident profiles, assigned room numbers, emergency contact records, and one-touch emergency inspection actions.

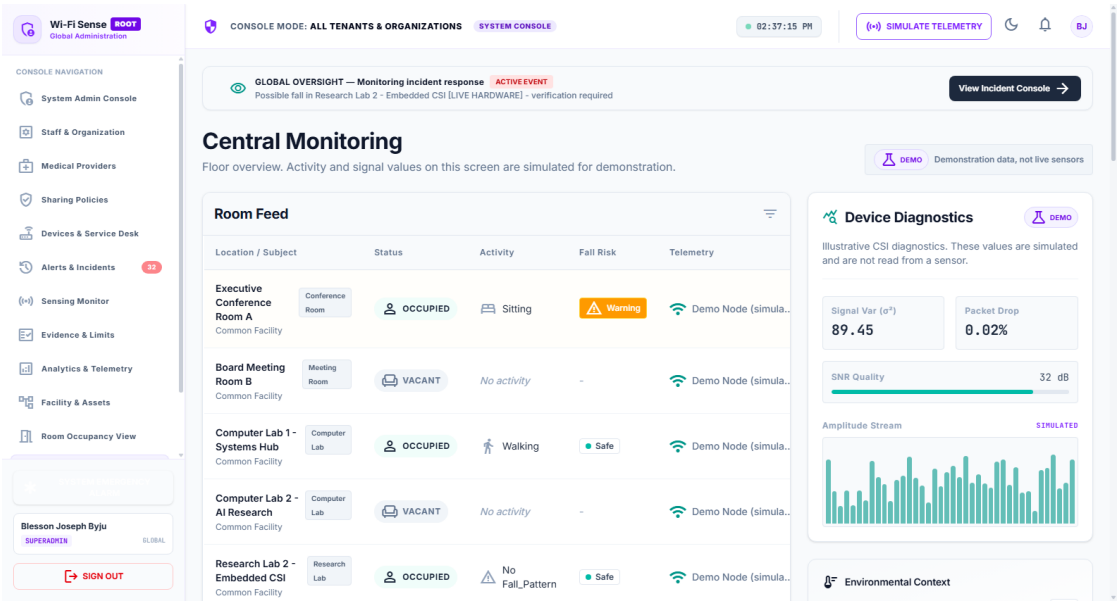


Figure 9.8: Multi-tenant system administration console

The System Administration console provides global tenant oversight across elder care and corporate organizations, system metrics, telemetry simulation controls, and administrative audit logging.

9.3 GIT LOG

The commit history below was read from the main branch of the Wi-Fi Sense repository (https://github.com/Blessonjoseph07/WIFISENSE) with the git log command. It details the chronological progression of features, bug fixes, security hardening, and deployment commits:

Table 9.1: Git Commit History of the Wi-Fi Sense Repository

No.	Commit	Date	Author	Message
1	8b3b79e	2026-10-24	Blesson Joseph Byju	Merge pull request #35 from Blessonjoseph07/feat/dot-provenance-helpline-presence
2	4c32e72	2026-10-24	Student	feat(ui): quiet orange dots instead of DEMO labels, care home helpline, payment test account, presence-first live boards
3	017dfde	2026-10-24	Blesson Joseph Byju	Merge pull request #34 from Blessonjoseph07/feat/razorpay-checkout
4	f881091	2026-10-24	Student	feat(payments): Razorpay Orders + Checkout for the family Care Plan (test mode by default)
5	2184042	2026-10-23	Blesson Joseph Byju	feat: fix direct dashboard login for all roles and add chapter 5 system testing report (#33)
6	d7da6e2	2026-10-23	Blesson Joseph Byju	Merge pull request #32 from Blessonjoseph07/feat/occupancy-learner
7	a1f03b4	2026-10-23	Student	feat(live): Phase 2 in-room occupancy learner (empty / few / busy)
8	eb2088d	2026-10-23	Blesson Joseph Byju	Merge pull request #31 from Blessonjoseph07/fix/movement-fetch-robust
9	e3a97cd	2026-10-23	Student	fix(live): movement box survives dropped requests; warn when the baseline is clearly wrong
10	ba5af83	2026-10-23	Blesson Joseph Byju	Merge pull request #30 from Blessonjoseph07/fix/live-presence-first
11	d338358	2026-10-23	Student	fix(live): presence first, fall detection off for live hardware
12	e5c6c9a	2026-10-23	Blesson Joseph Byju	Merge pull request #29 from Blessonjoseph07/feat/live-movement-indicator
13	df02945	2026-10-23	Student	feat(live): movement indicator for LIVE HARDWARE with an empty-room baseline
14	ab5b6ce	2026-10-23	Blesson Joseph Byju	Merge pull request #28 from Blessonjoseph07/feat/live-plain-summary
15	15cf8b0	2026-10-23	Student	feat(ui): plain-language summary for LIVE HARDWARE
16	263b833	2026-10-23	Blesson Joseph Byju	Merge pull request #27 from Blessonjoseph07/fix/hosting-stability

17	760f617	2026-10-23	Student	fix(hosting): harden Postgres connections, plain-language replay summary, presentation preflight
18	9806fa7	2026-10-23	Blesson Joseph Byju	Merge pull request #26 from Blessonjoseph07/fix/replay-resilient
19	39f74b5	2026-10-23	Student	fix(replay): retry gateway errors and add --skip to resume
20	6e92360	2026-10-23	Blesson Joseph Byju	Merge pull request #25 from Blessonjoseph07/fix/replay-check-device-source
21	7614811	2026-10-23	Student	fix(replay): stop at once when the device is not a dataset device
22	58cdb9e	2026-10-23	Blesson Joseph Byju	Merge pull request #24 from Blessonjoseph07/fix/show-device-token
23	88779b3	2026-10-23	Student	fix(devices): show the device token once after registering a node
24	711c9ec	2026-10-23	Blesson Joseph Byju	Merge pull request #23 from Blessonjoseph07/feat/room-picker-grouping
25	c1ea3f6	2026-10-23	Student	feat(devices): group the room picker by organization and add a data-source choice
26	32500c1	2026-10-23	Blesson Joseph Byju	Merge pull request #22 from Blessonjoseph07/fix/family-facility-name
27	d61f575	2026-10-23	Student	fix(family): show the facility's real name instead of a hard-coded St. Mary's
28	b79d6f3	2026-10-23	Blesson Joseph Byju	Merge pull request #21 from Blessonjoseph07/chore/render-minimal-seed
29	4d1e0bf	2026-10-23	Student	chore(render): back to the minimal seed profile (one user per role)
30	03a0295	2026-10-23	Blesson Joseph Byju	Merge pull request #20 from Blessonjoseph07/chore/render-full-seed
31	c944a32	2026-10-23	Student	chore(render): seed the full demo dataset on the hosted demo
32	49219bc	2026-10-23	Blesson Joseph Byju	Merge pull request #19 from Blessonjoseph07/feat/postgres-retention-ingest
33	34f2173	2026-10-23	Student	ci: keep the required check name 'Backend Boundary & Regression Tests' as a gate over the DB matrix
34	879d42e	2026-10-23	Student	Merge master into feat/postgres-retention-ingest
35	3f4c6ec	2026-10-23	Blesson Joseph Byju	Merge pull request #18 from Blessonjoseph07/demo/all-features
36	9125d08	2026-10-23	Student	Merge demo/all-features into feat/postgres-retention-ingest
37	86ca6d2	2026-10-23	Student	test: disable demo seeding in the production Settings() check
38	d2b995c	2026-10-23	Student	Merge demo/all-features into feat/postgres-retention-ingest
39	8660ff6	2026-10-23	Student	fix(config): check SECRET_KEY and

				database safeguards before the demo-seed guard
40	3e962cf	2026-10-23	Student	feat: PostgreSQL support, raw CSI retention, faster reported ingest
41	e2262d1	2026-10-23	Student	fix(config): demo seeding is opt-in in production (default off), keeping the SEED_PASSWORD guard for explicit opt-in
42	1b8c194	2026-10-23	Student	Merge branch 'feat/hosting-config' into demo/all-features
43	3dba7a3	2026-10-23	Student	Merge branch 'feat/evidence-status-page' into demo/all-features
44	61b6f4f	2026-10-23	Student	Merge branch 'fix/registration-form-crash' into demo/all-features
45	16fcb16	2026-10-23	Student	Merge branch 'feat/emergency-protocol-doctor-notify' into demo/all-features
46	92de9ae	2026-10-23	Student	Merge branch 'feat/replay-evaluation-visuals' into demo/all-features
47	40c5f05	2026-10-23	Student	feat(hosting): Render/Vercel config, safe demo seeding (SEED_PROFILE, SEED_PASSWORD) and psycopg2
48	9ca7f5e	2026-10-23	Student	feat: Evidence & Limits page and a public /health endpoint
49	27b2f8c	2026-10-23	Student	fix(auth): registration page no longer crashes (remove the dead Role Type dropdown)
50	65a8c78	2026-10-23	Student	fix(family): restore dataSource so the family portal renders